

UNIT- IV

Memory-Management Strategies: Introduction, Contiguous memory allocation, Paging, Structure of the Page Table, Swapping.

Virtual Memory Management: Introduction, Demand paging, Copy-on-write, Page replacement, Allocation of frames, Thrashing

Storage Management: Overview of Mass Storage Structure, HDD Scheduling.

Memory Management:

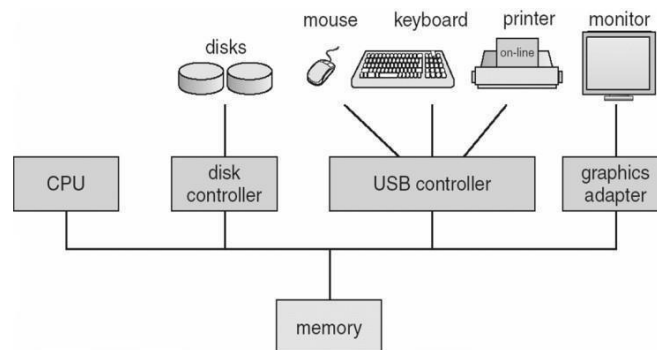
Introduction

Memory is central to the operation of a Modern Computer System.

Memory consists of a large array of words or bytes, each with its own address. The CPU fetches instructions from memory according to the value of the program counter. These instructions may cause additional loading from and storing to specific memory addresses.

A Typical Instruction-Execution Cycle:

- Fetches an instruction from memory.
- Instruction is then decoded and may cause operands to be fetched from memory.
- After the instruction has been executed on the operands, results may be stored back in memory.



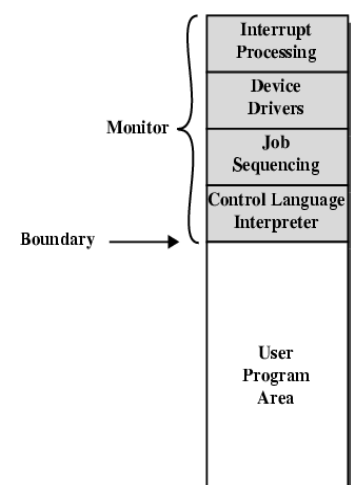
Memory unit sees only a stream of memory addresses; it does not know how they are generated (by the instruction counter, indexing, indirection, literal addresses, and so on) or what they are for (instructions or data).

Accordingly, we can ignore how a program generates a memory address.

Basically memory is divided into two parts:

- Resident Monitor
- User

Resident Monitor is a program used to provide automatic Job sequencing



Basic Hardware

Main memory and the Registers built into the processor itself are the only storage that the CPU can access directly.

Registers that are built into the CPU are generally accessible within one cycle of the CPU clock.

Main Memory is accessed via a transaction on the memory bus. Memory access may take many cycles of the CPU clock to complete, in which case the processor normally needs to stall(stopped), since it does not have the data required to complete the instruction that it is executing. This situation is intolerable because of the frequency of memory accesses

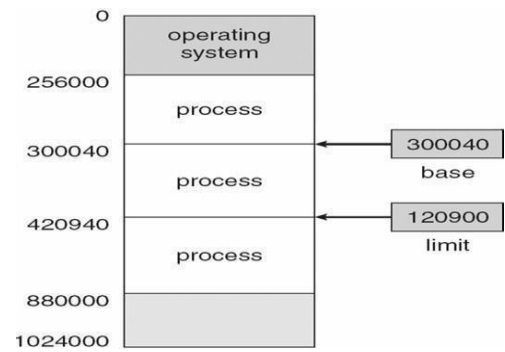


Fig: A base and a limit register define a logical address space.

The solution is to add *fast memory* between the CPU and main memory.

A memory buffer used to accommodate a speed differential, called a Cache., is used.

Not only relative speed of accessing physical memory, but we also must ensure correct operation, has to protect the operating system from access by user processes and, in addition, to protect user processes from one another. This protection must be provided by the hardware.

We can provide this protection by using two registers, usually a base and a limit, as illustrated in Figure

- Base register holds the smallest legal physical memory address;
- Limit register specifies the size of the range.

Ex:

If the base register holds 300040 and

limit register is 120900,

then the program can legally access all addresses from 300040 through 420940 (inclusive).

Protection of memory space is accomplished by having the CPU hardware compare *even/* address generated in user mode with the registers. Any attempt by a program executing in user mode to access operating-system memory or other users' memory results in a trap to the operating system, which treats the attempt as a fatal error

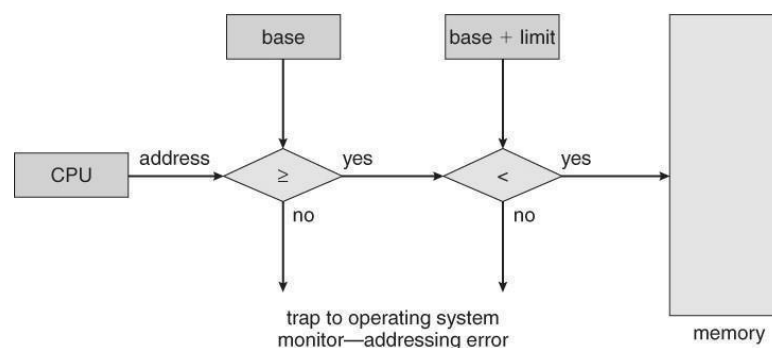


FIG: Hardware address protection with base and limit registers.

This scheme prevents a user program from (accidentally or deliberately) modifying the code or data structures of either the operating system or other users.

The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction. Since privileged instructions can be executed only in kernel mode, and

since only the operating system executes in kernel mode, only the operating system can load the base and limit registers.

Address Binding

Usually, a program resides on a disk as a binary executable file.

To be executed, the program must be brought into memory and placed within a process.

Depending on the memory management in use, the process may be moved between disk and memory during its execution.

The processes on the disk that are waiting to be brought into memory for execution form the *Input Queue*.

The normal procedure is to select one of the processes in the Input Queue and to load that process into memory. As the process is executed, it accesses instructions and data from memory. Eventually, the process terminates, and its memory space is declared available.

Most systems allow a user process to reside in any part of the physical memory.

Although the address space of the computer starts at 00000, the first address of the user process need not be 00000.

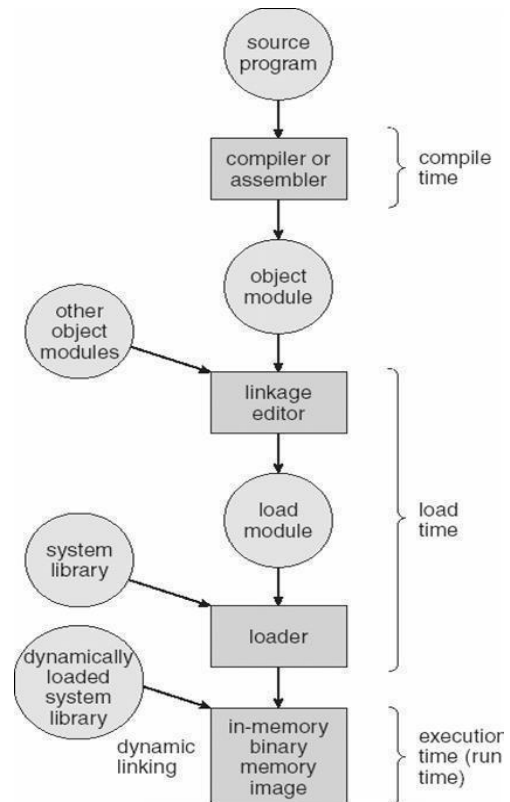


FIG: Multistep process of User Program

In most cases, a user program will go through several steps before being executed.

Each binding is a mapping from one address space to another.

Binding of instructions and data to memory addresses can be done at any step:

- **Compile time:** If you know at compile time where the process will reside in memory, then absolute code can be generated.

Ex: If you know that a user process will reside starting at location R , then the generated compiler code will start at that location and extend up from there.

If, at some later time, the starting location changes, then it will be necessary to recompile this code.

- **Load time:** If it is not known at compile time where the process will reside in memory, then the compiler must generate relocatable code.

In this case, final binding is delayed until load time.

If the starting address changes, we need only reload the user code to incorporate this changed value.

- **Execution time:** If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.

Logical Versus Physical Address Space

An address generated by the CPU is referred to as a *Logical Address or Virtual Address*

An address seen by the memory unit—that is, the one loaded into the memory-address register of the memory—is commonly referred to as a *Physical Address*.

The set of all logical addresses generated by a program is a *Logical Address Space*.

The set of all physical addresses corresponding to these logical addresses is a *Physical Address Space*.

The run-time mapping from virtual to physical addresses is done by a hardware device called the *Memory-Management Unit (MMU)*.

Note: Thus, in the execution-time address-binding scheme, the logical and physical address spaces differ.

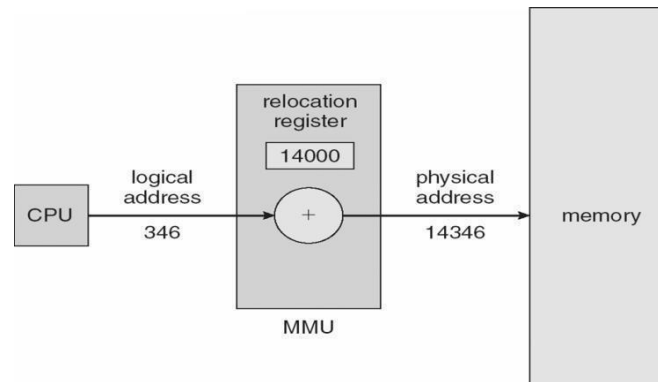


FIG: Dynamic Relocation using Relocation Register

The base register is now called a *Relocation Register*. The value in the relocation register is *added* to every address generated by a user process at the time it is sent to memory.

Ex: If the base is at 14000, then an attempt by the user to address location 0 is dynamically relocated to location 14000; an access to location 346 is mapped to location 14346.

The user program never sees the *real* physical addresses. The program can create a pointer to location 346, store it in memory, manipulate it, and compare it with other addresses—all as the number 346.

The user program deals with *logical* addresses.

Memory-Mapping hardware converts logical addresses into physical addresses.

We now have two different types of addresses:

- Logical addresses (in the range 0 to *max*) and
- Physical addresses (in the range $R + 0$ to $R + max$ for a base value R).

Dynamic Loading

To obtain better memory-space utilization, we can use Dynamic Loading.

With Dynamic Loading, a routine is not loaded until it is called. All routines are kept on disk in a relocatable load format. The main program is loaded into memory and is executed.

When a routine needs to call another routine, the calling routine first checks to see whether the other routine has been loaded. If not, the relocatable linking loader is called to load the desired routine into memory and to update the program's address tables to reflect this change. Then control is passed to the newly loaded routine.

Advantage: Unused routine is never loaded. This method is particularly useful when large amounts of code are needed to handle infrequently occurring cases, such as error routines.

Dynamic Linking and Shared Libraries

Dynamic Linking is similar to that of Dynamic Loading. Linking is postponed until execution time.

With Dynamic Linking, a *stub* is included in the image for each library routine reference.

The stub is a small piece of code that indicates

- how to locate the appropriate memory-resident library routine or
- how to load the library if the routine is not already present.

When the stub is executed, it checks to see whether the needed routine is already in memory. If not, the program loads the routine into memory. Stub replaces itself with the address of the routine and executes the routine.

Under this scheme, all processes that use a language library execute only one copy of the library code. A library may be replaced by a new version, and all programs that reference the library will automatically use the new version. Without dynamic linking, all such programs would need to be relinked to gain access to the new library.

Thus, only programs that are compiled with the new library version are affected by the incompatible changes incorporated in it. Other programs linked before the new library was installed will continue using the older library. This system is also known as *Shared Libraries*.

Dynamic linking generally requires help from the operating system.

Swapping

Process must be in memory to be executed. Process can be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution.

Ex: Assume a multiprogramming environment with a *Round-Robin CPU-scheduling algorithm*.

When a quantum expires, the memory manager will start to swap out the process that just finished and to swap another process into the memory space that has been freed

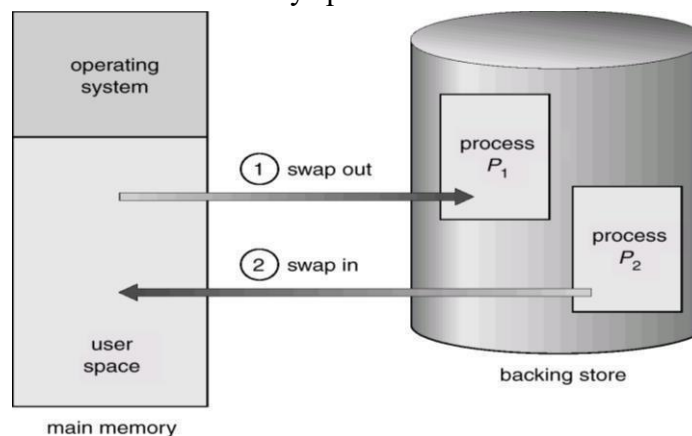


FIG: Swapping of two processes using a disk as a Backing store

In the meantime, the CPU scheduler will allocate a time slice to some other process in memory. When each process finishes its quantum, it will be swapped with another process.

Memory manager can swap processes fast enough that some processes will be in memory, ready to execute, when the CPU scheduler wants to reschedule the CPU. In addition, the quantum must be large enough to allow reasonable amounts of computing to be done between swaps.

A variant of this swapping policy is used for *Priority-Based Scheduling Algorithms*. If a higher-priority process arrives and wants service, the memory manager can swap out the lower-priority process and then load and execute the higher-priority process. When the higher-priority process

finishes, the lower-priority process can be swapped back in and continued. This variant of swapping is sometimes called *Roll Out, Roll In*.

Backing Store:

Swapping requires a *Backing Store*. The backing store is commonly a fast disk.

It must be large enough to accommodate copies of all memory images for all users, and it must provide direct access to these memory images.

System maintains a *Ready Queue* consisting of all processes whose memory images are on the backing store or in memory and are ready to run.

Dispatcher:

Whenever the CPU scheduler decides to execute a process, it calls the *Dispatcher*.

The dispatcher checks to see whether the next process in the queue is in memory. If it is not, and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired process. It then reloads registers and transfers control to the selected process.

Ex: The context-switch time in such a swapping system is fairly high.

To get an idea of the context-switch time, let us assume that the user process is 10 MB in size and the backing store is a standard hard disk with a transfer rate of 40 MB per second.

The actual transfer of the 10-MB process to or from main memory takes

$$10000 \text{ KB} / 40000 \text{ KB per second} = 1/4 \text{ second} = 250 \text{ milliseconds.}$$

Assuming that no head seeks are necessary, and assuming an average latency of 8 milliseconds, the swap time is 258 milliseconds. Since we must both swap out and swap in, the total swap time is about 516 milliseconds.

For efficient CPU utilization, we want the execution time for each process to be long relative to the swap time.

Swapping is constrained by other factors:

- If we want to swap a process, we must be sure that it is completely idle.
- Of particular concern is any pending I/O.
- A process may be waiting for an I/O operation when we want to swap that process to free up memory.
- If the I/O is asynchronously accessing the user memory for I/O buffers, then the process cannot be swapped.

Contiguous Memory Allocation

The main memory must accommodate both the operating system and the various user processes.

The memory is usually divided into two partitions:

- One for the resident operating system and
- One for the user processes.

We can place the operating system in either low memory or high memory.

Several user processes reside in memory at same time. So, we need to consider how to allocate available memory to the processes that are in the input queue waiting to be brought into memory.

In this Contiguous Memory Allocation, each process is contained in a single contiguous section of memory.

Memory Mapping and Protection

Relocation register contains the value of the smallest physical address;

Limit register contains the range of logical addresses

Ex; Relocation register = 100040 and Limit register = 74600.

With relocation and limit registers, each logical address must be less than the limit register; the MMU maps the logical address *dynamically* by adding the value in the relocation register.

This mapped address is sent to memory

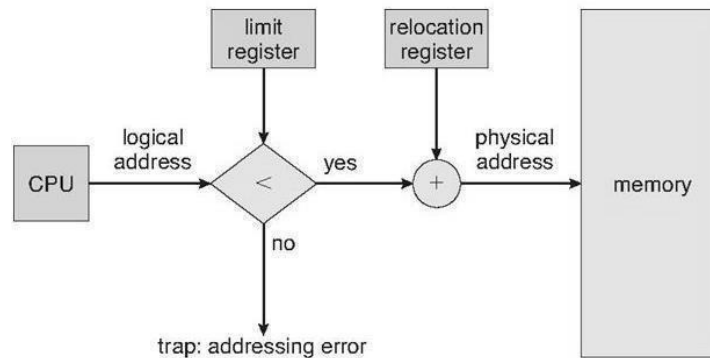


FIG: Hardware support for Relocation and Limit Registers

When the CPU scheduler selects a process for execution, the dispatcher loads the relocation and limit registers with the correct values.

Because every address generated by the CPU is checked against these registers, we can protect both the operating system and the other users' programs and data from being modified by this running process.

Relocation-Register scheme provides an effective way to allow the operating-system size to change dynamically. This flexibility is desirable in many situations.

Memory Allocation

One of the simplest methods for allocating memory is to divide memory into several fixed-sized partitions.

Each partition may contain exactly one process. Thus, the Degree of Multiprogramming is bound by the number of partitions.

In this *Multiple Partition Method*, when a partition is free, a process is selected from the input queue and is loaded into the free partition.

- When the process terminates, the partition becomes available for another process. This method (called MFT); it is no longer in use. It is used by IBM OS/360 operating system
- The method described next is a generalization of the Fixed-Partition scheme (called MVT); it is used primarily in batch environments.

In the *Fixed-Partition Scheme*, the Operating System keeps a table indicating which parts of memory are available and which are occupied.

Initially, all memory is available for user processes and is considered one large block of available memory, a *Hole*.

When a process arrives and needs memory, the system searches the set for a hole that is large enough for this process. If we find one, we allocate only as much memory as is needed, Keeping the rest available to satisfy future requests

As processes enter the system, they are put into an Input queue. Operating System takes into account the memory requirements of each process and the amount of available memory space in determining which processes are allocated memory.

When process is allocated space, it is loaded into memory and it can then compete for the CPU.

When a process terminates, it releases its memory, which the Operating System may then fill with another process from the Input queue

At any given time, we have a list of available block sizes and the input queue. Operating System can order the input queue according to a scheduling algorithm. Memory is allocated to process until, the memory requirement of the next process cannot be satisfied

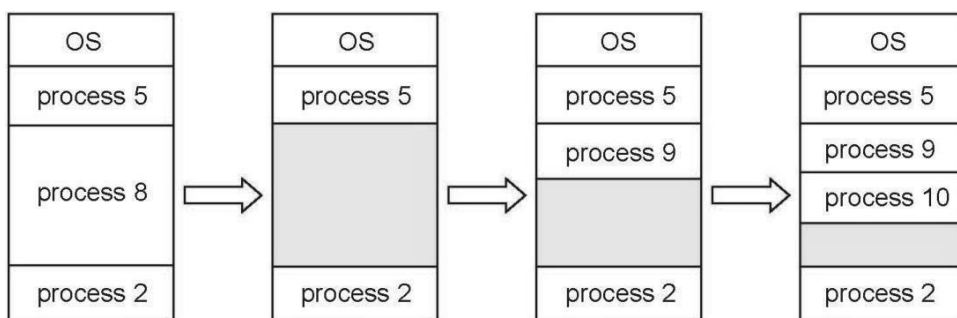
That is, no available block of memory(or hole) is large enough to hold that process. Operating System can then wait until a large enough block is available, or it can skip down the Input queue to see whether the small memory requirements of some other process can met

Generally, at any given time we have a set of holes of various sizes scattered throughout memory. When a process arrives and needs memory, the system searches the set for a hole that is large enough for this process

If the hole is too large, it is split into two parts:

- One part is allocated to the arriving process;
- Other is returned to the set of holes.

When a process terminates, it releases its block of memory, which is then placed back in the set of holes. If the new hole is adjacent to other holes, these adjacent holes are merged to form one larger hole.



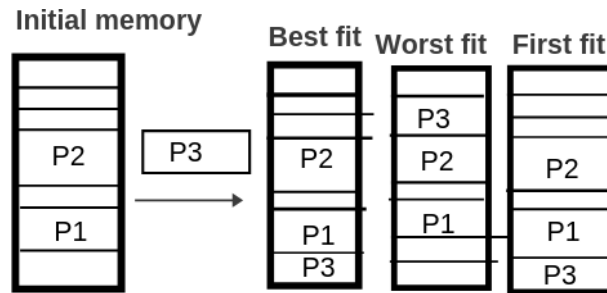
There are many solutions to this problem most commonly used to select a free hole from the set of available holes.

- First-Fit
- Best-Fit And
- Worst-Fit Strategies

First fit: Allocate the *first* hole that is big enough. Searching can start either at the beginning of the set of holes or where the previous first-fit search ended. We can stop searching as soon as we find a free hole that is large enough.

Best fit: Allocate the *smallest* hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produces the smallest leftover hole.

Worst fit: Allocate the *largest* hole. Again, we must search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.



Example, given 5 memory partitions of 100 KB, 500 KB, 200 KB, 300 KB, 600 KB (in-order). How would each of first-fit, best-fit & worst-fit algorithms place processes of 212 KB, 417 KB, 112 KB, 426 KB (in-order). Which algorithm makes the most efficient use of memory?

First-fit

212 KB is put in 500 KB partition.

417 KB is put in 600 KB partition.

112 KB is put in 288 KB partition. (New partition 288 KB = 500 KB - 212 KB)

426 KB must wait.

Best-fit

212 KB is put in 300 KB partition.

417 KB is put in 500 KB partition.

112 KB is put in 200 KB partition.

426 KB is put in 600 KB partition.

Worst-fit

212 KB is put in 600 KB partition.

417 KB is put in 500 KB partition.

112 KB is put in 300 KB partition.

426 KB must wait.

In this example, Best-fit turns out to be the efficient.

Note:

- Simulations show that either first or best fit are better than worst fit in terms of both time and storage utilization.
- First and best fits are about equal in terms of storage utilization, but first fit is faster.



Best Fit Allocation in OS



Block 2 results in the least memory wastage

P1 goes to Block 2



Block 3 results in the least memory wastage

P2 goes to Block 3



Block 5 results in the least memory wastage

P3 goes to Block 5



Block 1 results in the least memory wastage

P4 goes to Block 1

Block / Size	Available	Can Occupy P1?	Memory Wastage After Process Occupies
Block 1 (100K)	Yes	Yes	$100 - 40 = 60$
Block 2 (50K)	Yes	Yes	$50 - 40 = 10$ Best
Block 3 (30K)	Yes	No	-
Block 4 (120K)	Yes	Yes	$120 - 40 = 80$
Block 5 (35K)	Yes	No	-

Block / Size	Available	Can Occupy P2?	Memory Wastage After Process Occupies
Block 1 (100K)	Yes	Yes	$100 - 10 = 90$
Block 2 (50K)	No	-	-
Block 3 (30K)	Yes	Yes	$30 - 10 = 20$ Best
Block 4 (120K)	Yes	Yes	$120 - 10 = 110$
Block 5 (35K)	Yes	Yes	$35 - 10 = 25$

Block / Size	Available	Can Occupy P3?	Memory Wastage After Process Occupies
Block 1 (100K)	Yes	Yes	$100 - 30 = 70$
Block 2 (50K)	No	-	-
Block 3 (30K)	Yes	-	-
Block 4 (120K)	Yes	Yes	$120 - 30 = 90$
Block 5 (35K)	Yes	Yes	$35 - 30 = 5$ Best

Block / Size	Available	Can Occupy P4?	Memory Wastage After Process Occupies
Block 1 (100K)	Yes	Yes	$100 - 60 = 40$ Best
Block 2 (50K)	No	-	-
Block 3 (30K)	Yes	-	-
Block 4 (120K)	Yes	Yes	$120 - 60 = 60$
Block 5 (35K)	Yes	-	-



First Fit Allocation in OS



Block / Size	Available	Can Occupy P1?	Memory Wastage After Process Occupies
Block 1 (20K)	Yes	No	
Block 2 (100K)	Yes	Yes	$100 - 90 = 10$
Block 3 (40K)	Yes	No	
Block 4 (200K)	Yes	No	
Block 5 (10K)	Yes	No	

Block 2 is the first to Fit P1
P1 goes to Block 2



Block / Size	Available	Can Occupy P2?	Memory Wastage After Process Occupies
Block 1 (20K)	Yes	No	
Block 2 (100K)	No	No	
Block 3 (40K)	Yes	No	
Block 4 (200K)	Yes	Yes	$200 - 50 = 150$
Block 5 (10K)	Yes	No	

Block 4 is the first to Fit P2
P2 goes to Block 4



Block / Size	Available	Can Occupy P3?	Memory Wastage After Process Occupies
Block 1 (20K)	Yes	No	
Block 2 (100K)	No	No	
Block 3 (40K)	Yes	Yes	$40 - 30 = 10$
Block 4 (200K)	No	No	
Block 5 (10K)	Yes	No	

Block 3 is the first to Fit P3
P3 goes to Block 3



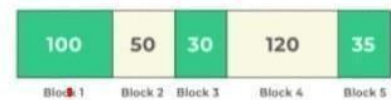
Block / Size	Available	Can Occupy P4?	Memory Wastage After Process Occupies
Block 1 (20K)	Yes	No	
Block 2 (100K)	No	No	
Block 3 (40K)	No	No	
Block 4 (200K)	No	No	
Block 5 (10K)	Yes	No	

None of the available blocks can accommodate P4. It remains unallocated.



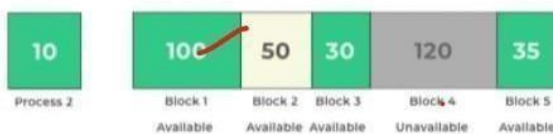


Worst Fit Allocation in OS Example



	Available	Can Occupy P1?	Memory Wastage After Process Occupies
Block 1	Yes	Yes	$100 - 40 = 60$
Block 2	Yes	Yes	$50 - 40 = 10$
Block 3	Yes	No	-
Block 4	Yes	Yes	$120 - 40 = 80$ Worst
Block 5	Yes	No	-

P1 goes to Block 4



	Available	Can Occupy P2?	Memory Wastage After Process Occupies
Block 1	Yes	Yes	$100 - 10 = 90$ Worst
Block 2	Yes	Yes	$50 - 10 = 40$
Block 3	Yes	Yes	$30 - 10 = 20$
Block 4	No	-	-
Block 5	Yes	Yes	$35 - 10 = 25$

P2 goes to Block 1



	Available	Can Occupy P3?	Memory Wastage After Process Occupies
Block 1	No	-	-
Block 2	Yes	Yes	$50 - 30 = 20$ Worst
Block 3	Yes	Yes	$30 - 30 = 0$
Block 4	No	-	-
Block 5	Yes	Yes	$35 - 30 = 5$

P3 goes to Block 2



	Available	Can Occupy P4?	Memory Wastage After Process Occupies
Block 1	No	-	-
Block 2	No	-	-
Block 3	Yes	No	-
Block 4	No	-	-
Block 5	Yes	No	-

P4 remains unallocated, As can not be fit by any Free blocks

Multiple Partitions of Memory:

In order to facilitate multi programming, memory is partitioned into multiple partitions for the various processes which must run concurrently.

Two memory management schemes were popular:

- Multiple Contiguous Fixed Partition (MFP)
- Multiple Contiguous Variable Partition (MVP)

MFP is implemented by the multiprogramming with fixed number of tasks (MFT) strategy.

MVP is implemented by the multiprogramming with variable number of tasks (MVT) strategy.

Multiprogramming with Fixed number of Tasks

In multiprogramming with a fixed number of tasks (MFT), all jobs entering the system are put in a job queue. The scheduler allocates memory to jobs based on the size of the job. Once loaded, the job can then compete for CPU time.

Following is an account of how memory is allocated to jobs, with illustrations in figures

- Classify jobs according to their memory requirement (user may be required to specify memory requirements, or the system may determine it)
- Keep all jobs in one queue. The scheduler selects the next job to run and waits on large enough memory space to fit it. Allocation may be based on best-fit, or first-fit. Best-Fit starts with the largest job and fits it in the smallest available space, until all possible allocations are made. It could also process the jobs in FIFO order, fitting each in the smallest space available.

First-fit processes the jobs in FIFO order, fitting each in the first available space, until no more allocation is possible.

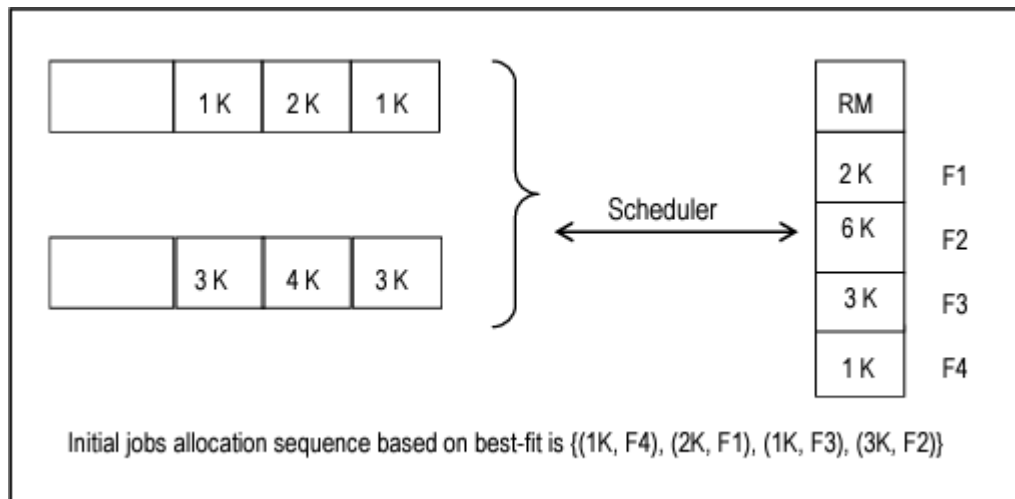


Fig: Illustrating MFT with Separate Queues based on Job Size

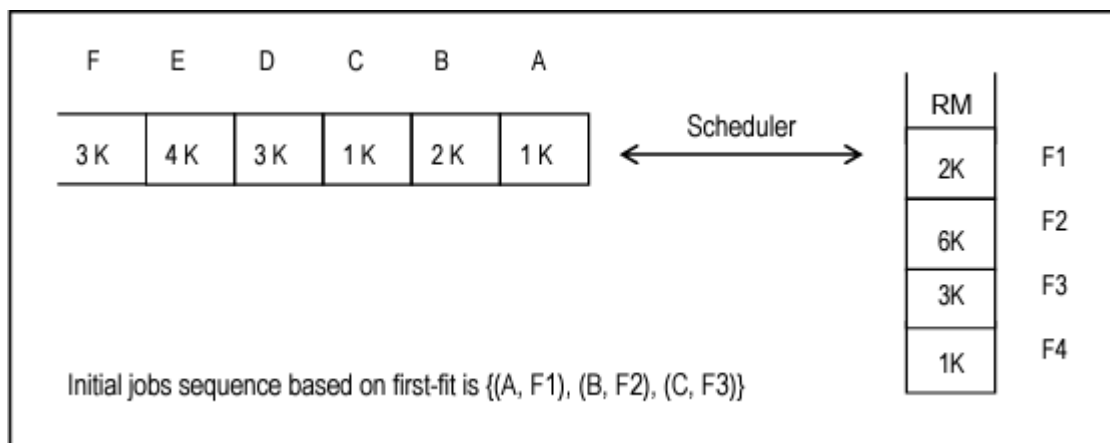


Fig: Illustrating MFT with One Queue

Advantages of MFT:

- Simple
- A fixed number of jobs at a time and this is known
- No frequent switching of boundary registers
- Fences are fixed
- The OS is catered for a number of PCB's
- The strategy can be used along with swapping.

Disadvantages of MFT:

- Job scheduling is an expensive overhead
- Fragmentation can be high:
- Internal fragmentation occurs when space is allocated but not used
- External fragmentation occurs when space not allocated and not used
- There could be a job that is too large to run in the system (for any of the partitions)

Multiprogramming with Variable number of Tasks

Multiprogramming with variable number of tasks (MVT), which was successfully implemented on UNIVAC 1108 machines, allows partition sizes to vary dynamically.

As many jobs as can fit in memory are loaded. Initially, memory is one large hole. As jobs arrive, just enough space for each job is allocated, thus gradually decreasing the size of the hole.

When a job terminates, its space is released as a hole. Adjacent holes are merged into a simple (larger) hole. Holes are allocated based on best-fit, first-fit or worst-fit.

First-fit processes the jobs FIFO, trying to fit each job in the first available hole.

Best-fit starts with the largest job and fits it in the smallest available hole. This continues until there are no jobs that can fit.

Worst-fit starts with the smallest job and fits it in the largest available hole. This continues until there are no remaining jobs that can fit.

As an example, consider space of 216K and five jobs as shown in figure

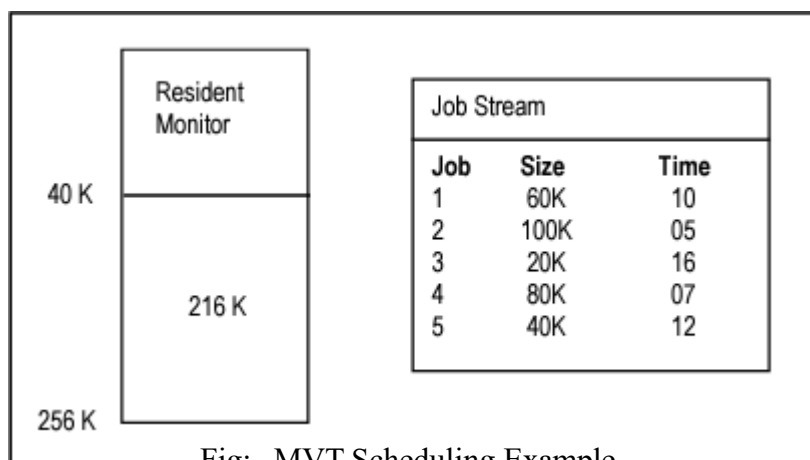


Fig: MVT Scheduling Example

Assuming first-fit allocation, figure below shows how the jobs might be allocated.

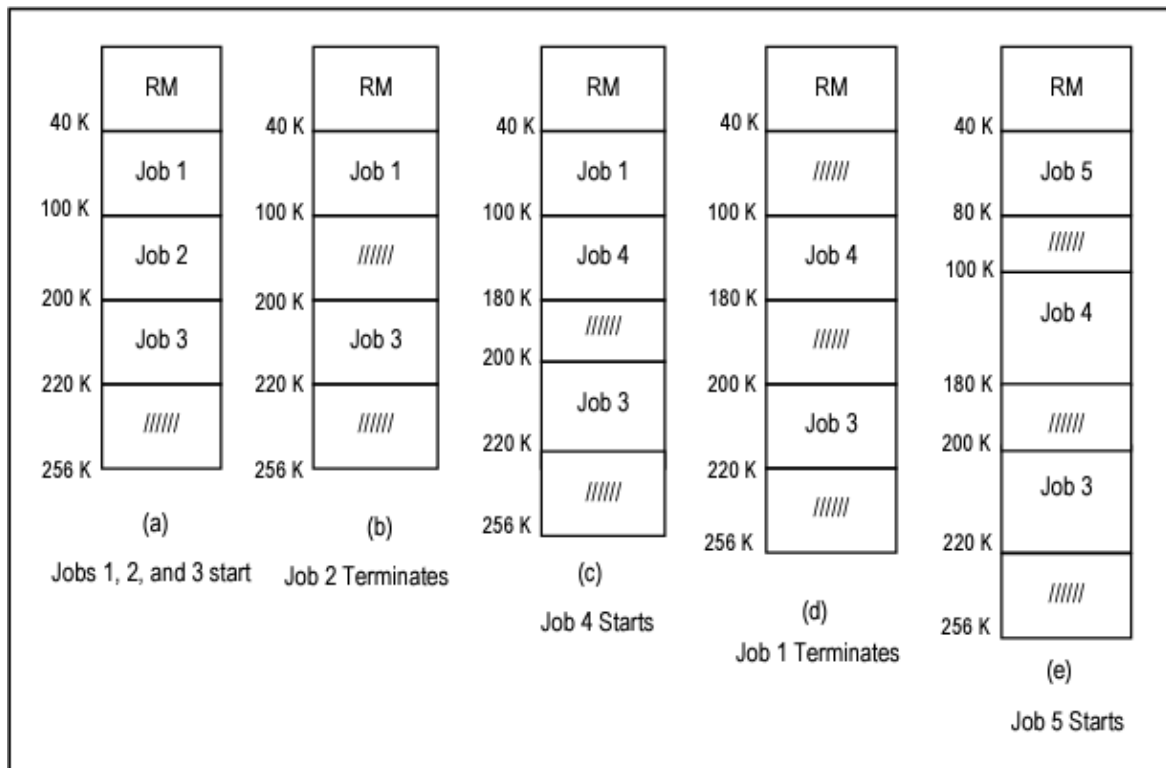


Fig:MVT Illustrating Memory Allocation for the Job Stream for above Fig

Advantages of MVT:

- Significant reduction of internal fragmentation (to be confined to the last block occupied by a job).
- More efficient use of memory space.
- The strategy can be used along with swapping.
- With dynamic relocation, a job can be rolled to a different location than originally occupied.

Drawbacks of MVT:

- External fragmentation is still possible, though its likelihood is also reduced.
- To offset this, compaction gathers all scattered holes into one large hole.
- Another strategy is to hold waiting jobs for a time quantum, and allow jobs in memory to clear up; then continue.
- Starvation can occur if a large job is at the head of the queue. Skipping circumvents this problem.

Differences between MFT and MVT:

MFT caters for a fixed number of jobs while MVT caters for a variable number of jobs.

MFT uses fixed fences while MVT uses variable fences.

Fragmentation is a major problem with MFT; with MVT, internal fragmentation is significantly reduced and external fragmentation can be corrected.

Fragmentation

In Contiguous Memory Allocation, Allocation when the processes come into RAM, Space is allocated to them. As the process gets loaded and removed from the memory these spaces get broken into small pieces of memory that it can't be allocated to the coming processes. This problem is known as *Fragmentation*

Two factors:

- Whether we are using the First-fit or Best-fit strategy can affect the amount of Fragmentation
- Which end of free block is allocated

Depending on the total amount of memory storage and the average process size, External Fragmentation may be minor or a major problem

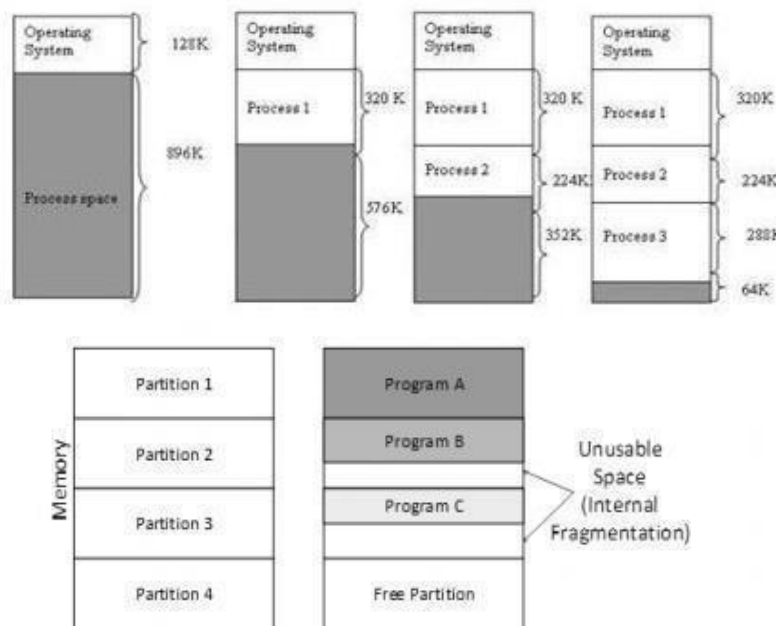
50-percent Rule:

One-third of memory may be unusable. This property is known as 50-percent Rule

Memory Fragmentation can be:

- External Fragmentation
- Internal Fragmentation

Internal Fragmentation: Allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.



Consider the figure above where a fixed sized memory allocation mechanism is being followed. Initially, the memory is empty and the allocator has divided the memory into fixed size partitions. Then later three programs named A, B, C have been loaded to the first three partitions while the 4th partition is still free. Program A matches the size of the partition, so there is no wastage in that partition, but Program B and Program C are smaller than the partition size. So in partition 2 and partition 3 there is remaining free space. However, this free space is unusable as the memory allocator only assigns full partitions to programs but not parts of it. This wastage of free space is called *Internal Fragmentation*.

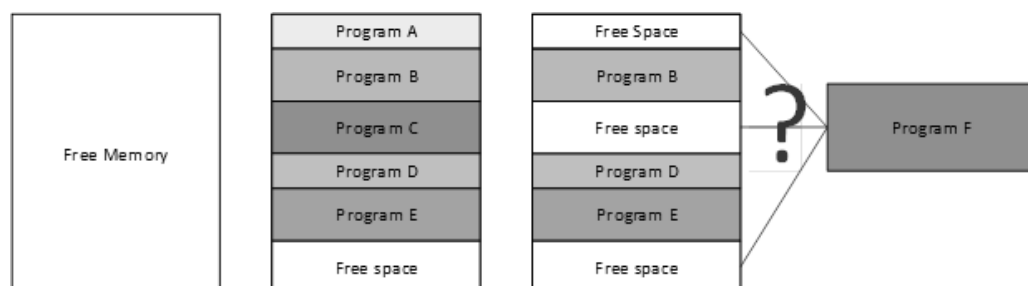
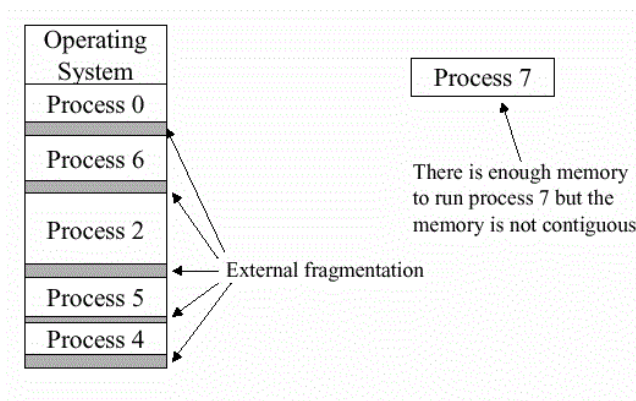
In the above example, it is equal sized fixed partitions but this can even happen in a situation where partitions of various fixed sizes are available. Usually the memory or hardest space is divided into blocks that are usually the size of powers of 2 such as 2, 4, 8, 16 bytes. So a program or a file of 3 bytes will be assigned to a 4 byte block but one byte of that block will become unusable causing internal fragmentation.

Systems with Fixed-size partitions units such as single partitions scheme and paging suffers from Internal Fragmentation

External Fragmentation: Total memory space exists to satisfy a request, but available spaces are not contiguous.

Storage is fragmented into a large number of small holes. This fragmentation problem can be severe.

In the worst case, we could have a block of free (or wasted) memory between every two processes. If all these small pieces of memory were in one big free block instead, we might be able to run several more processes.



Consider the figure above where memory allocation is done dynamically.

In Dynamic Memory Allocation, the allocator allocates only the exact needed size for that program. First memory is completely free. Then the Programs A, B, C, D and E of different sizes are loaded one after the other and they are placed in memory contiguously in that order. Then later, Program A and Program C closes and they are unloaded from memory. Now there are three free space areas in the memory, but they are not adjacent. Now a large program called Program F is going to be loaded but neither of the free space block is not enough for Program F. The addition of all the free spaces is definitely enough for Program F, but due to the lack of adjacency that space is unusable for Program F. This is called *External Fragmentation*.

Both the first-fit and best-fit strategies for memory allocation suffer from external fragmentation.

As processes are loaded and removed from memory, the free memory space is broken into little pieces.

Systems with variable size allocation units such as multiple partitions scheme and segmentation suffers from External Fragmentation

Solutions:

One solution to the problem of external fragmentation is *Compaction*.

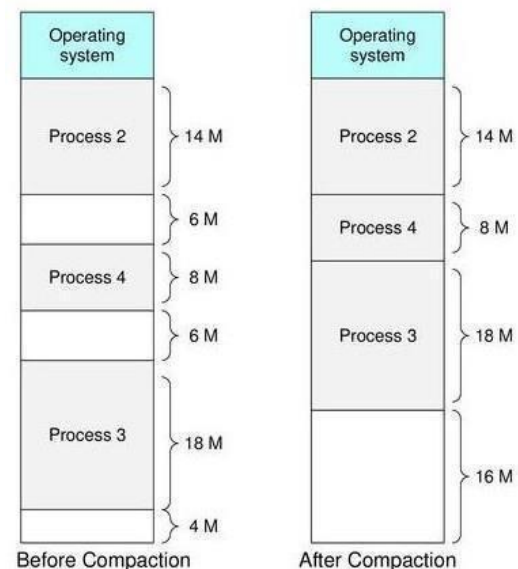
Compaction: The goal is to shuffle the memory contents so as to place all free memory together in one large block.

Compaction is not always possible. If relocation is static and is done at assembly or load time, compaction cannot be done; compaction is possible *only* if relocation is dynamic and is done at execution time.

When compaction is possible, we must determine its cost. The simplest compaction algorithm is to move all processes toward one end of memory; all holes move in the other direction, producing one large hole of available memory. This scheme can be expensive. Another possible solution to the external-fragmentation problem is to permit the logical address space of the processes to be noncontiguous, thus allowing a process to be allocated physical memory wherever the latter is available.

Two complementary techniques achieve this solution:

- Paging
- Segmentation

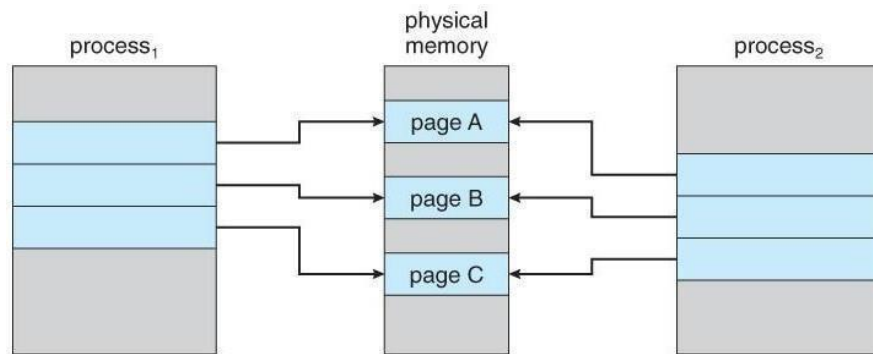


Copy-on-Write

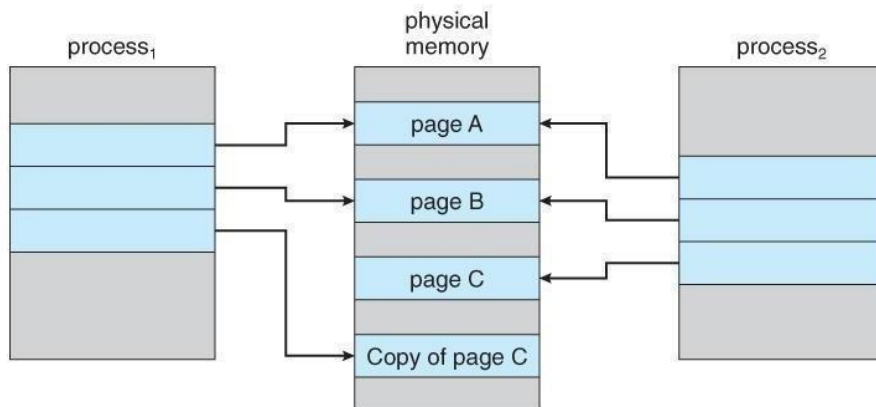
Copy on Write (CoW) is a resource management technique.

The main use of Copy on Write is in the implementation of the fork system call in which it shares the virtual memory (pages) of the OS.

When a parent process creates a child process then both of these processes initially will share the same pages in memory. These shared pages will be marked as copy-on-write which means that if any of these processes will try to modify the shared pages then only a copy of these pages will be created. And the modifications will be done on the copy of pages by that process and thus not affecting the other process.



Before process 1 modifies Page C



After process 1 modifies Page C

Pages used to satisfy copy-on-write duplications are typically allocated using *zero-fill-on-demand*, meaning that their previous contents are zeroed out before the copy proceeds.

The CoW technique is used by several Operating systems like Linux, Solaris, and Windows XP.

The CoW technique is an efficient process creation technique as only the pages that are modified are copied.

Paging:

Paging is a memory-management scheme that permits the physical address space of a process to be non contiguous.

Paging avoids the considerable problem of fitting memory chunks of varying sizes onto the backing store

Basic Method

The basic method for implementing paging involves

- Breaking Physical Memory into fixed-sized blocks called *Frames* and
- Breaking Logical Memory into blocks of the same size called *Pages*.

When a process is to be executed, its pages are loaded into any available memory frames from the backing store.

Backing store is divided into fixed-sized blocks that are of the same size as the memory frames.

Hardware Support for paging is illustrated in Figure

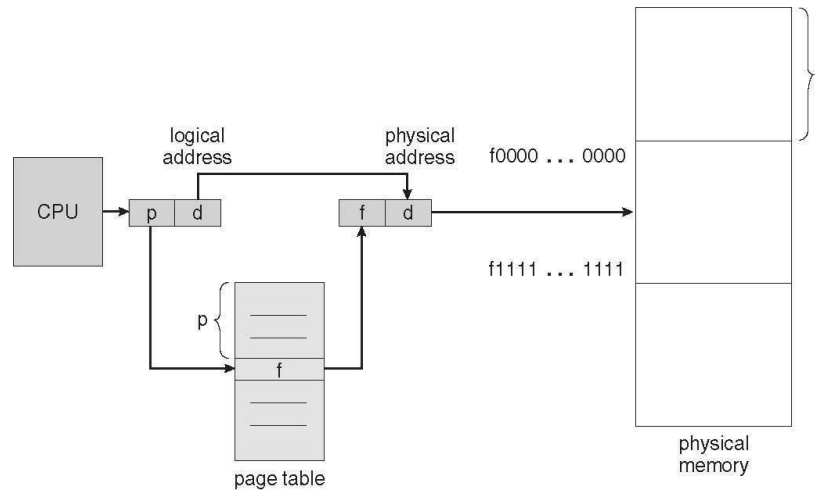


FIG: Paging Hardware

Every address generated by the CPU is divided into two parts:

- Page Number (P) and
- Page Offset (D).

Page Number is used as an index into a page table. The Page Table contains the base address of each page in physical memory.

This base address is combined with the Page Offset to define the physical memory address that is sent to the memory unit.

The Paging Model of Memory is as shown:

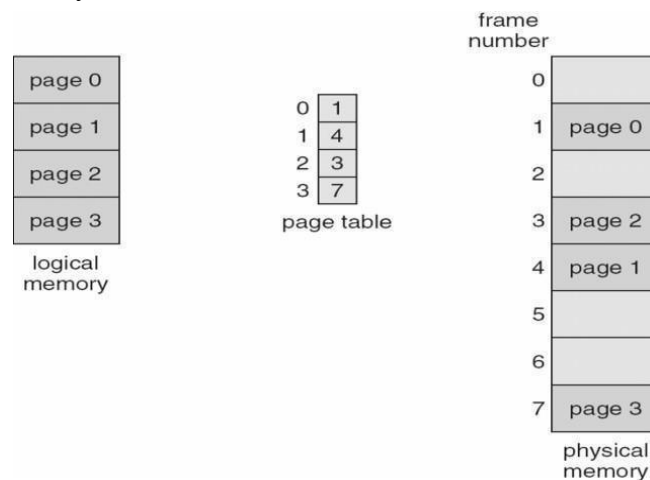


Figure: Paging model of memory

The page size (like the frame size) is defined by the hardware.

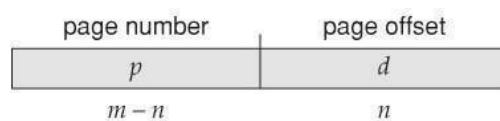
The size of a page is typically a power of 2, varying between 512 bytes and 16 MB per page, depending on the computer architecture.

If the

size of logical address space is 2^m and

a page size is 2^n addressing units (bytes or words), then the high-order $m - n$ bits of a logical address designate the page number, and the n low-order bits designate the page offset.

Thus, the logical address is as follows:



where p is an index into the page table and d is the displacement within the page.

Ex: Consider the memory in below Figure

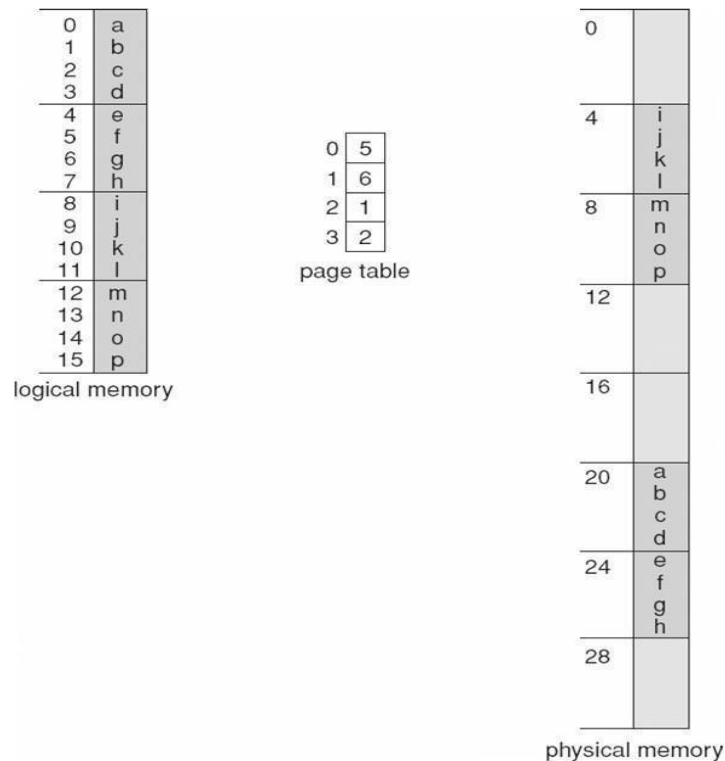


Figure: Paging example for a 32-byte memory with 4-byte pages.

Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages), we show how the user's view of memory can be mapped into physical memory.

Logical address 0 is page 0, offset 0.

Indexing into the page table, we find that page 0 is in frame 5.

Thus, Logical address 0 maps to physical address 20 ($= (5 \times 4) + 0$).

Logical address 3 (page 0, offset 3) maps to physical address 23 ($= (5 \times 4) + 3$).

Logical address 4 is page 1, offset 0; according to the page table, page 1 is mapped to frame 6.

Thus, logical address 4 maps to physical address 24 ($= (6 \times 4) + 0$).

Logical address 13 maps to physical address 9.

Here paging itself is a form of dynamic relocation. Every logical address is bound by the paging hardware to some physical address.

When a process arrives in the system to be executed, its size, expressed in pages, is examined. Each page of the process needs one frame. Thus, if the process requires n pages, at least n frames must be available in memory.

If n frames are available, they are allocated to this arriving process. The first page of the process is loaded into one of the allocated frames, and the frame number is put in the page table for this process. The next page is loaded into another frame, and its frame number is put into the page table, and so on (Fig)

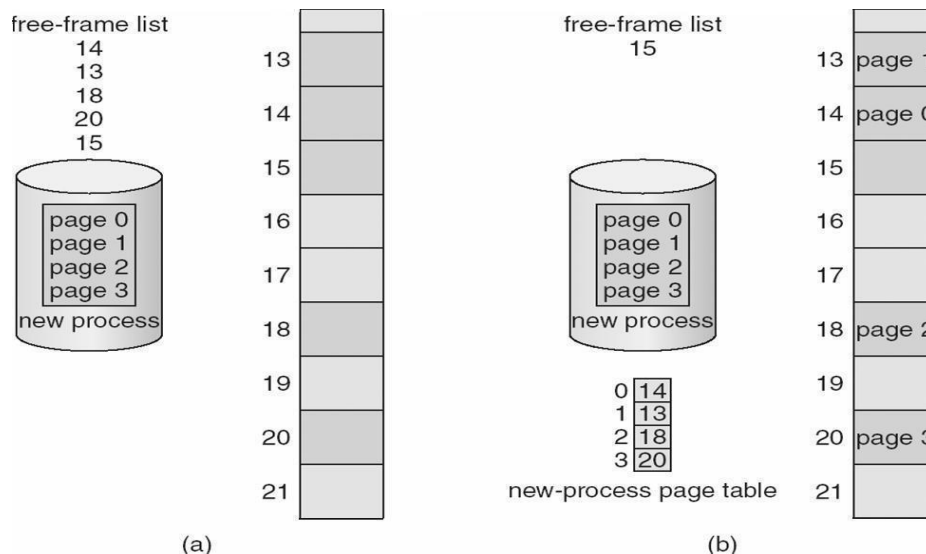


Figure: Free frames (a) Before allocation and (b) After allocation.

An important aspect of paging is the clear separation between the user's view of memory and the actual physical memory.

User program views memory as one single space, containing only this one program.

In fact, the user program is scattered throughout physical memory, which also holds other programs.

Difference between the user's view of memory and the actual physical memory is reconciled by the address-translation hardware.

Logical addresses are translated into physical addresses. This mapping is hidden from the user and is controlled by the operating system

Hardware Support

Each operating system has its own methods for storing page tables.

Most allocate a page table for each process. A pointer to the page table is stored with the other register values (like the instruction counter) in the Process Control Block.

When the dispatcher is told to start a process, it must reload the user registers and define the correct hardware page-table values from the stored user page table.

Hardware implementation of the page table can be done in several ways.

In the simplest case, the page table is implemented as a set of dedicated registers. These registers should be built with very high-speed logic to make the paging-address translation efficient

Most contemporary computers, however, allow the page table to be very large.

For these machines, the use of fast registers to implement the page table is not feasible. Rather, the page table is kept in main memory, and a Page-Table Base Register (PTBR) points to the page table.

Changing page tables requires changing only this one register, substantially reducing context-switch time.

Problem with this approach: Time required to access a user memory location.

The standard solution to this problem: Use a special, small, fastlookup hardware cache, called a Translation Look-Aside Buffer (TLB). The TLB is associative, high-speed memory.

Each entry in the TLB consists of two parts:

- a key (or tag) and

- a value.

When the Associative Memory is presented with an item, the item is compared with all keys simultaneously.

If the item is found, the corresponding value field is returned. The search is fast; the hardware, however, is expensive.

Typically, the number of entries in a TLB is small, often numbering between 64 and 1,024.

The TLB is used with page tables in the following way. The TLB contains only a few of the page-table entries.

When a logical address is generated by the CPU, its page number is presented to the TLB.

- If the page number is found, its frame number is immediately available and is used to access memory.
- If the page number is not in the TLB (known as a TLB miss), a memory reference to the page table must be made.

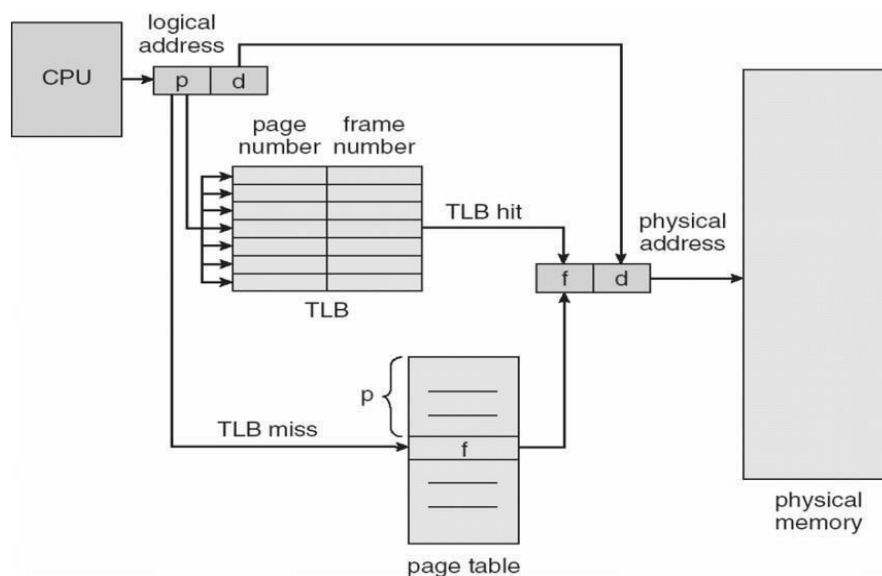


Figure: Paging hardware with TLB.

In addition, we add the page number and frame number to the TLB, so that they will be found quickly on the next reference. If the TLB is already full of entries, the operating system must select one for replacement.

An ASID uniquely identifies each process and is used to provide address-space protection for that process. When the TLB attempts to resolve virtual page numbers, it ensures that the ASID for the currently running process matches the ASID associated with the virtual page. If the ASIDs do not match, the attempt is treated as a *TLB miss*.

Hit Ratio: The percentage of times that a particular page number is found in the TLB

An 80-percent hit ratio means that we find the desired page number in the TLB 80 percent of the time.

Ex: If it takes 20 nanoseconds to search the TLB and 100 nanoseconds to access memory, then a mapped-memory access takes 120 nanoseconds when the page number is in the TLB.

If we fail to find the page number in the TLB (20 nanoseconds), then we must first access memory for the page table and frame number (100 nanoseconds) and then access the desired byte in memory (100 nanoseconds), for a total of 220 nanoseconds.

To find the Effective Memory-Access Time, we weight each case by its probability:

$$\text{Effective Access Time} = 0.80 \times 120 + 0.20 \times 220 = 140 \text{ nanoseconds.}$$

In this example, we suffer a 40-percent slowdown in memory-access time (from 100 to 140 nanoseconds).

For a 98-percent hit ratio, we have

$$\text{Effective Access Time} = 0.98 \times 120 + 0.02 \times 220 = 122 \text{ nanoseconds.}$$

This increased hit rate produces only a 22 percent slowdown in access time.

Ex: Compute effective access time (EAT) for 70% hit ratio, 20 ns (nano seconds) to search TLB and 100 ns (nano seconds) to access memory. Observe the difference when it is changed to 90% hit ratio.

Solution:

For 70% hit ratio,

$$P_1 = 70\% = 0.70$$

$$\text{Hit memory time} = \text{Access time of TLB} + \text{Access time of main memory} = 20 + 100 = 120 \text{ ns}$$

$$\text{Miss memory time} = \text{Access time of TLB} + ((L+1) * \text{Access time of main memory})$$

$$= 20 + ((1+1) * 100) = 220 \text{ ns}$$

Here, L – Number of levels of page table

$$\text{EAT (effective access time)} = P_1 \times \text{hit memory time} + (1-P_1) \times \text{miss memory time}$$

$$= 0.70 \times 120 + 0.30 \times 220 = 150 \text{ nanoseconds.}$$

Here, we suffer a 50 percent slowdown in average memory access time.

For 90% hit ratio,

$$P_2 = 90\% = 0.90$$

$$\text{Hit memory time} = \text{Access time of TLB} + \text{Access time of main memory} = 20 + 100 = 120 \text{ ns}$$

$$\text{Miss memory time} = \text{Access time of TLB} + ((L+1) * \text{Access time of main memory})$$

$$= 20 + ((1+1) * 100) = 220 \text{ ns}$$

Here, L – Number of levels of page table

$$\text{EAT (effective access time)} = P_2 \times \text{hit memory time} + (1-P_2) \times \text{miss memory time}$$

$$= 0.90 \times 120 + 0.10 \times 220 = 130 \text{ nanoseconds.}$$

Here, due to the increased hit ratio we suffer only 30 percent slowdown in average memory access time.

Protection

Memory protection in a paged environment is accomplished by protection bits associated with each frame. These bits are kept in the page table.

One bit can define a page to be read-write or read-only.

Every reference to memory goes through the page table to find the correct frame number. At the same time that the physical address is being computed, the protection bits can be checked to verify that no writes are being made to a read-only page. An attempt to write to a read-only page causes a hardware trap to the operating system (or memory-protection violation).

One additional bit is generally attached to each entry in the page table: a *Valid-Invalid Bit*.

- When this bit is set to "valid," the associated page is in the process's logical address space and is thus a legal (or valid) page.
- When the bit is set to "invalid," the page is not in the process's logical address space.

Operating System sets this bit for each page to allow or disallow access to the page.

Ex: In a system with a 14-bit address space (0 to 16383), we have a program that should use only addresses 0 to 10468. Given a page size of 2 KB, we get the situation shown in Figure.

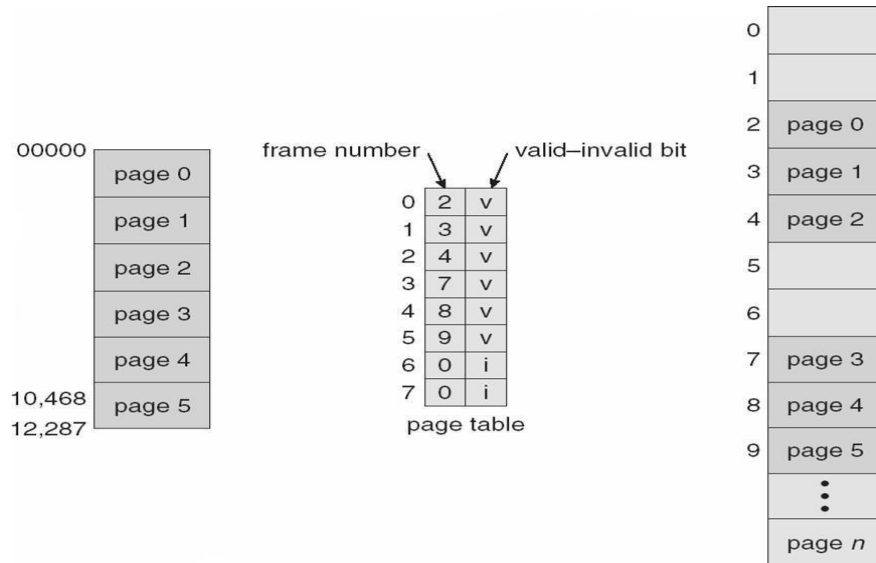


Figure: Valid (v) or Invalid (i) bit in a page table.

Addresses in pages 0,1, 2,3, 4, and 5 are mapped normally through the page table. Any attempt to generate an address in pages 6 or 7, however, will find that the valid-invalid bit is set to invalid, and the computer will trap to the operating system (invalid page reference).

Notice that this scheme has created a problem. Because the program extends to only address 10468, any reference beyond that address is illegal.

However, references to page 5 are classified as valid, so accesses to addresses up to 12287 are valid. Only the addresses from 12288 to 16383 are invalid. This problem is a result of the 2-KB page size and reflects the internal fragmentation of paging.

Shared Pages

An advantage of paging is the possibility of *sharing* common code.

This consideration is particularly important in a time-sharing environment.

Ex:

Consider a system that supports 40 users, each of whom executes a text editor.

If the text editor consists of 150 KB of code and 50 KB of data space, we need 8,000 KB to support the 40 users.

If the code is reentrant code (or pure code), however, it can be shared, as shown in Figure.

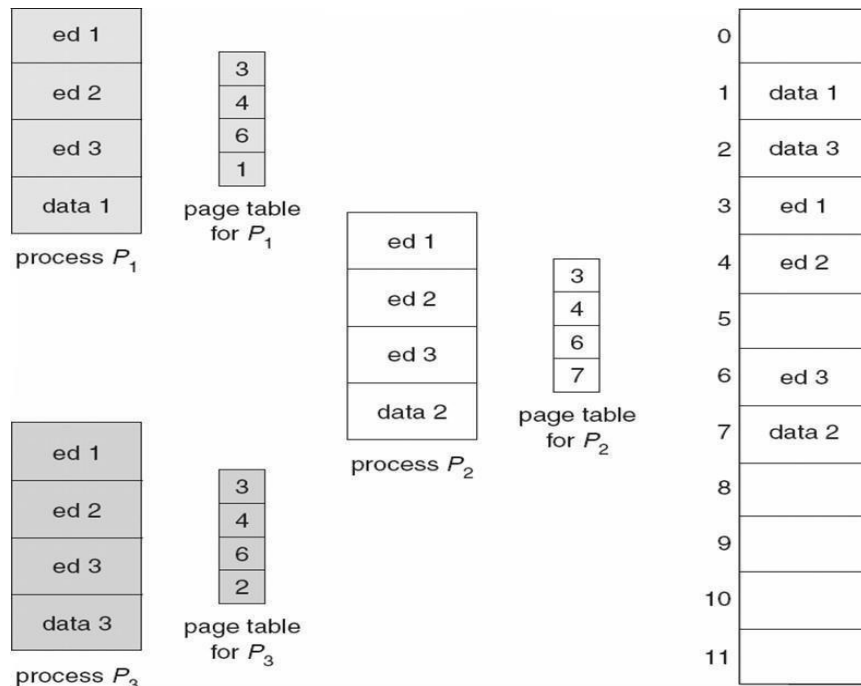


Figure Sharing of code in a paging environment.

Here we see a three-page editor—each page 50 KB in size (the large page size is used to simplify the figure)—being shared among three processes.

Each process has its own data page. *Reentrant Code* is non-self-modifying code; it never changes during execution.

Thus, two or more processes can execute the same code at the same time. Each process has its own copy of registers and data storage to hold the data for the process's execution. The data for two different processes will, of course, be different.

Only one copy of the editor need be kept in physical memory. Each user's page table maps onto the same physical copy of the editor, but data pages are mapped onto different frames. Thus, to support 40 users, we need only one copy of the editor (150 KB), plus 40 copies of the 50 KB of data space per user.

The total space required is now 2,150 KB instead of 8,000 KB—a significant savings.

The sharing of memory among processes on a system is similar to the sharing of the address space of a task by threads.

Advantages of Paging:

- Paging eliminates fragmentation.
- Support higher degree of multiprogramming.
- Paging increases memory and processor utilization.

Disadvantages of Paging:

- Page address mapping hardware usually increases the cost of the computer.
- Memory must be used to store various tables like page table, memory map table etc.

Structure of the Page Table

Some of the most common techniques for structuring the page table.

Hierarchical Paging

Most modern computer systems support a large logical address space (2^{32} to 2^{64}).

In such an environment, the page table itself becomes excessively large.

Ex: Consider a system with a 32-bit logical address space. If the page size in such a system is 4 KB, then a page table may consist of up to 1 million entries.

Assuming that each entry consists of 4 bytes, each process may need up to 4 MB of physical address space for the page table alone. Clearly, we would not want to allocate the page table contiguously in main memory.

One simple solution to this problem is to divide the page table into smaller pieces. We can accomplish this division in several ways.

One way is to use a two-level paging algorithm, in which the page table itself is also paged as shown.

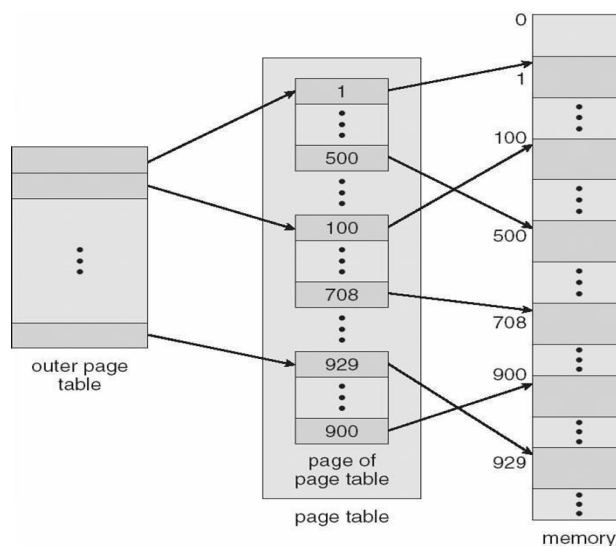
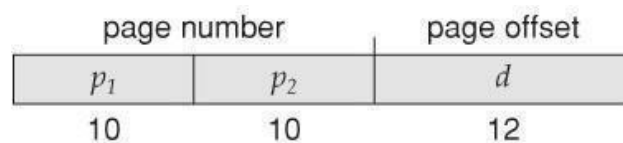


Figure: A Two-Level Page-Table Scheme.

In the example of a 32-bit machine memory with a page size of 4 KB. A logical address is divided into a page number consisting of 20 bits and a page offset consisting of 12 bits.

Because we page the page table, the page number is further divided into a 10-bit page number and a 10-bit page offset.

Thus, a logical address is as follows:



where p_1 is an index into the outer page table and p_2 is the displacement within the page of the outer page table. The address-translation method for this architecture is shown in Figure.

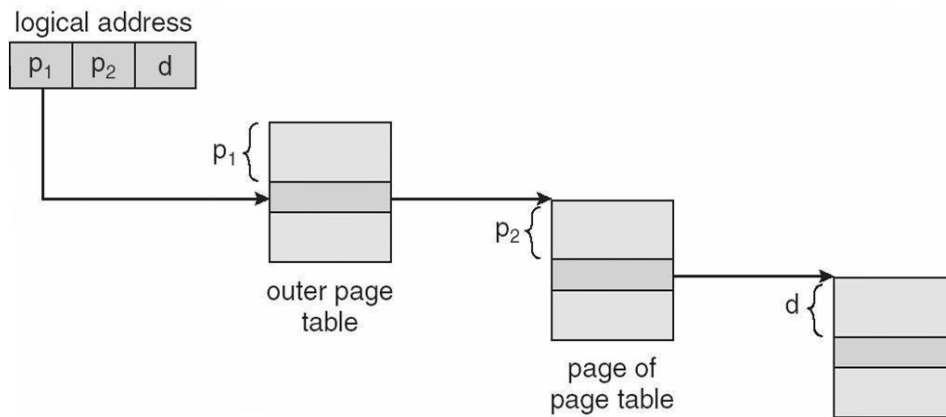


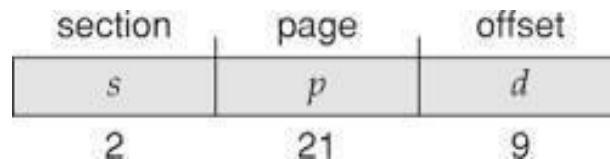
Figure: Address Translation For A Two-Level 32-Bit Paging Architecture.

Because address translation works from the outer page table inward, this scheme is also known as a forward-mapped page table.

The VAX architecture also supports a variation of two-level paging. The VAX is a 32-bit machine with a page size of 512 bytes. The logical address space of a process is divided into four equal sections, each of which consists of 2^{30} bytes.

Each section represents a different part of the logical address space of a process. The first 2 high-order bits of the logical address designate the appropriate section. The next 21 bits represent the logical page number of that section, and the final 9 bits represent an offset in the desired page.

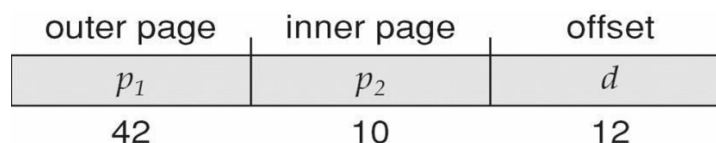
An address on the VAX architecture is as follows:



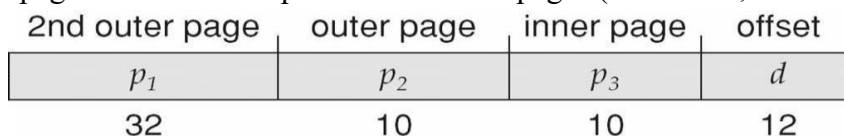
where s designates the section number, p is an index into the page table, and d is the displacement within the page.

Even when this scheme is used, the size of a one-level page table for a VAX process using one section is $221 \text{ bits} \times 4 \text{ bytes per entry} = 8 \text{ MB}$.

For a system with a 64-bit logical-address space, a two-level paging scheme is no longer appropriate. To illustrate this, let us suppose that the page size in such a system, is 4 KB (2^{12}). In this case, the page table consists of up to 2^{52} entries. If we use a two-level paging scheme, then the inner page tables can conveniently be one page long, or contain 2^{10} 4-byte entries. The addresses look like this:



The outer page table consists of 2^{42} entries, or 2^{44} bytes. We can divide the outer page table in various ways. We can page the outer page table, giving us a three-level paging scheme. Suppose that the outer page table is made up of standard-size pages (2^{10} entries, or 2^{12} bytes)



The SPARC architecture (with 32-bit addressing) supports a three-level paging scheme, whereas the 32-bit Motorola 68030 architecture supports a four-level paging scheme.

Hashed Page Tables

A common approach for handling address spaces larger than 32 bits is to use a hashed page table, with the hash value being the virtual page number.

Each entry in the hash table contains a linked list of elements that hash to the same location (to handle collisions).

Each element consists of three fields:

- Virtual page number,
- Value of the mapped page frame, and
- A pointer to the next element in the linked list.

The algorithm works as follows: The virtual page number in the virtual address is hashed into the hash table. The virtual page number is compared with field 1 in the first element in the linked list.

- If there is a match, the corresponding page frame (field 2) is used to form the desired physical address.
- If there is no match, subsequent entries in the linked list are searched for a matching virtual page number.

This scheme is shown in Figure.

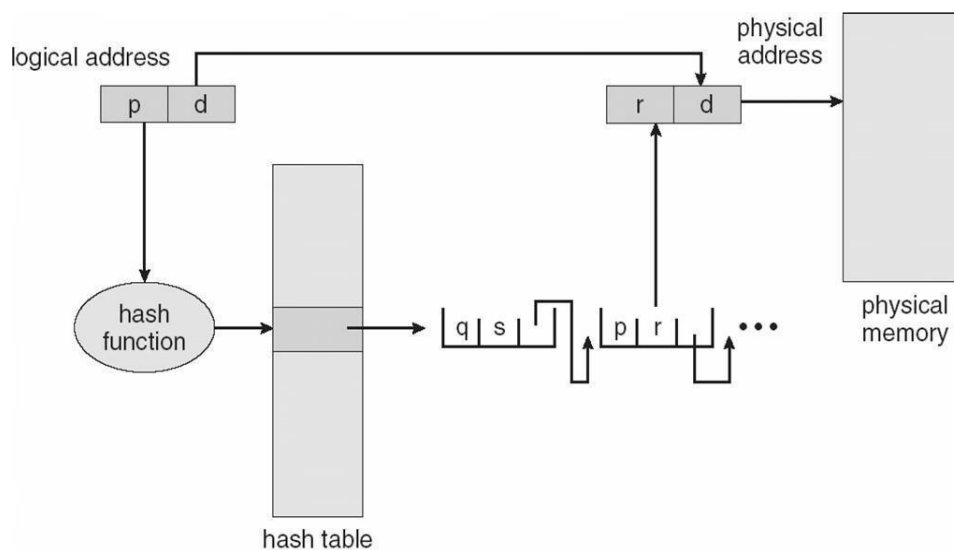


Figure: Hashed Page Table.

A variation of this scheme that is favourable for 64-bit address spaces has been proposed. This variation uses *Clustered Page Tables*, which are similar to hashed page tables except that each entry in the hash table refers to several pages (such as 16) rather than a single page.

Drawback: Each page table may consist of millions of entries. These tables may consume large amounts of physical memory just to keep track of how other physical memory is being used.

Inverted Page Tables

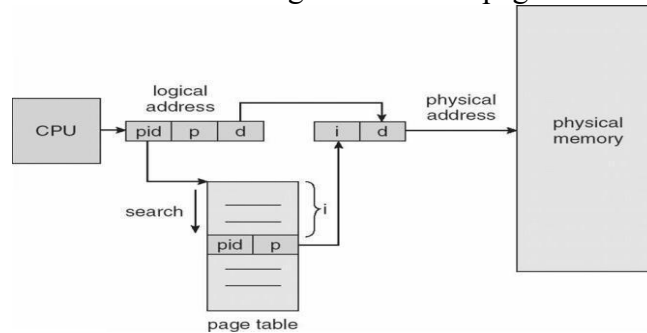
To solve the Drawback in Hashed Page Table, we can use an inverted page table.

An Inverted Page Table has one entry for each real page (or frame) of memory. Each entry consists of the virtual address of the page stored in that real memory location, with information

about the process that owns that page. Thus, only one page table is in the system, and it has only one entry for each page of physical memory.

Figure shows the operation of an Inverted Page Table.

Figure: Inverted page table.



Inverted page tables often require that an address-space identifier be stored in each entry of the page table, since the table usually contains several different address spaces mapping physical memory.

Each virtual address in the system consists of a triple

$$\langle \text{process} - id, \text{page} - number, \text{offset} \rangle.$$

Each inverted page-table entry is a pair $\langle \text{process} - id, \text{page} - number \rangle$ where the *process - id* assumes the role of the Address-Space Identifier.

When a memory reference occurs, part of the virtual address, consisting of

$$\langle \text{process} - id, \text{pagenumber} \rangle,$$

is presented to the memory subsystem. The inverted page table is then searched for a match.

- If a match is found—say at entry *i* — then the physical address $\langle i, \text{offset} \rangle$ is generated.
- If no match is found, then an illegal address access has been attempted.

Although this scheme decreases the amount of memory needed to store each page table, it increases the amount of time needed to search the table when a page reference occurs.

Disadvantage: Systems that use inverted page tables have difficulty implementing shared memory.

Virtual memory management

Background

It is a technique that allows the execution of processes that are not completely in memory.

Advantage: Programs can be larger than physical memory.

It abstracts main memory into an extremely large, uniform array of storage, separating logical memory as viewed by the user from physical memory.

Virtual memory also allows processes to share files easily and to implement shared memory.

The ability to execute a program that is only partially in memory would give many benefits:

- A program would no longer be constrained by the amount of physical memory that is available. Users would be able to write programs for an extremely large *virtual* address space, simplifying the programming task.
- Because each user program could take less physical memory, more programs could be run at the same time, with a corresponding increase in CPU utilization and throughput but with no increase in response time or turnaround time.
- Less I/O would be needed to load or swap each user program into memory, so each user program would run faster.

Virtual memory involves the separation of logical memory as perceived by users from physical memory. This separation, allows an extremely large virtual memory to be provided for programmers when only a smaller physical memory is available (Figure).

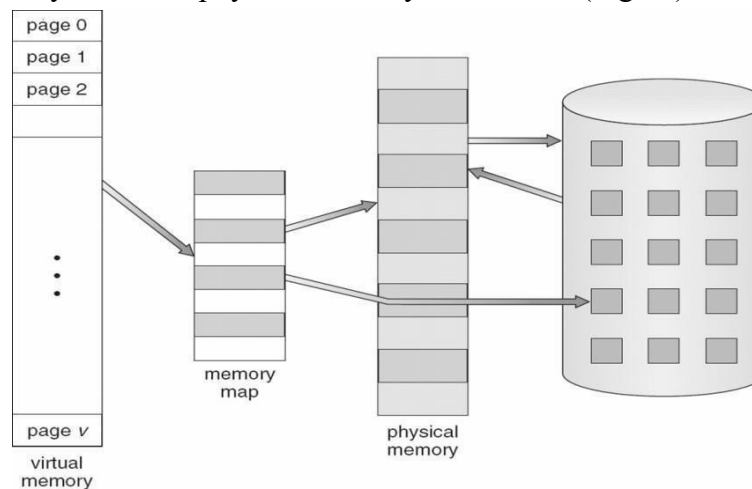


FIG: Virtual Memory that is Larger than Physical Memory

The *Virtual Address Space* of a process refers to the logical (or virtual) view of how a process is stored in memory.

Typically, this view is that a process begins at a certain logical address — say, address 0 — and exists in contiguous memory, as shown in Figure

It is up to the memory management unit (MMU) to map logical pages to physical page frames in memory.

In the above figure(FIG: *Virtual Address Space*) that we allow for the heap to grow upward in memory as it is used for dynamic memory allocation. The large blank space (or hole) between the heap and the stack is part of the virtual address space but will require actual physical pages only if the heap or stack grows.

Virtual address spaces that include holes are known as *Sparse Address Spaces*.

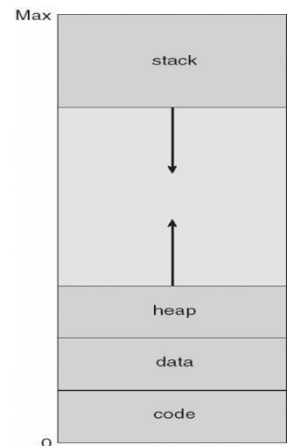


FIG: Virtual Address Space

In addition to separating logical memory from physical memory, virtual memory also allows files and memory to be shared by two or more processes through page sharing.

This leads to the following benefits:

- System libraries can be shared by several processes through mapping of the shared object into a virtual address space. Although each process(Figure: Shared library using Virtual Memory)
- Virtual memory enables processes to share memory.
- Virtual memory allows one process to create a region of memory that it can share with another process.
- Virtual memory can allow pages to be shared during process creation with the `fork()` system call, thus speeding up process creation.

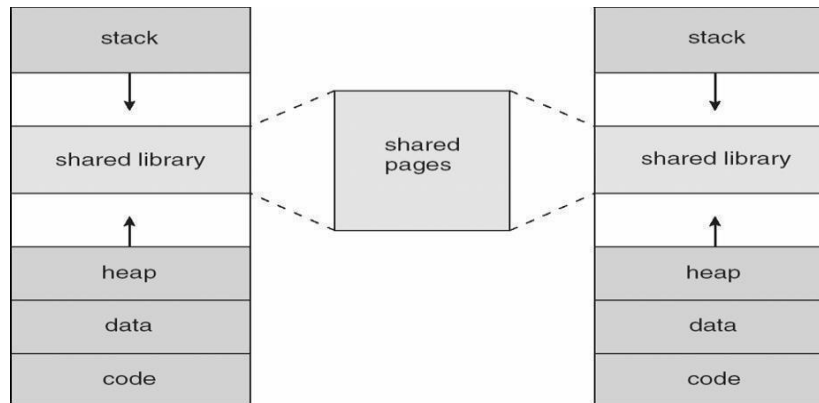


Figure: Shared library using Virtual Memory.

Advantages :

- Allocating memory is easy and cheap
- Any free page is ok, OS can take first one out of list it keeps
- Eliminates external fragmentation
- Data (page frames) can be scattered all over PM
- Pages are mapped appropriately anyway
- Allows demand paging and prepaging
- More efficient swapping
- No need for considerations about fragmentation
- Just swap out page least likely to be used

Disadvantages :

- Longer memory access times (page table lookup)
- Can be improved using TLB
- Guarded page tables
- Inverted page tables
- Memory requirements (one entry per VM page)
- Improve using Multilevel page tables and variable page sizes (super-pages)
- Guarded page tables
- Page Table Length Register (PTLR) to limit virtual memory size
- Internal fragmentation

Demand Paging

Problem:

Consider how an executable program might be loaded from disk into memory.

Sol 1: Load the entire program in physical memory at program execution time.

However, a problem with this approach, is that we may not initially *need* the entire program in memory.

Sol 2: An alternative strategy is to initially load pages only as they are needed by user. This technique is known as *Demand Paging* and is commonly used in virtual memory systems. With Demand-Paged Virtual Memory, pages are only loaded when they are demanded during program execution; pages that are never accessed are thus never loaded into physical memory.

A Demand-Paging System is similar to a Paging system with swapping (Figure) where processes reside in secondary memory (usually a disk).

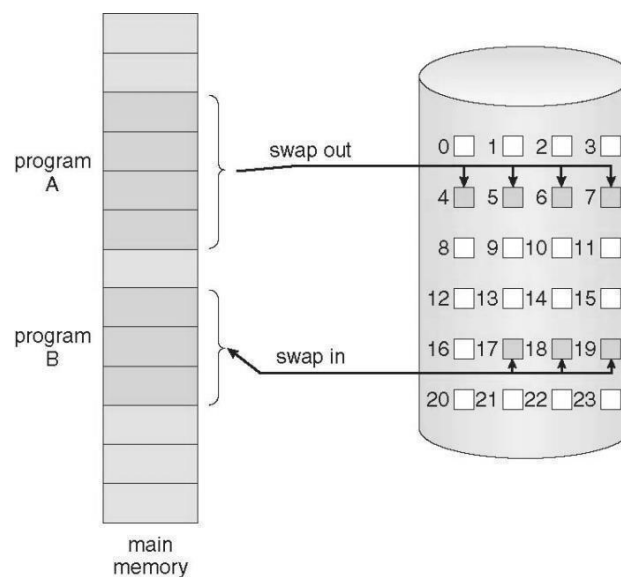


Figure: Transfer of a paged memory to contiguous disk space.

When we want to execute a process, we swap it into memory. Rather than swapping the entire process into memory, we use a *Lazy Swapper*.

A Lazy Swapper never swaps a page into memory unless that page will be needed.

Instead of using the term *swapper* technically, *Pager* is used.

A swapper manipulates entire processes, whereas a Pager is concerned with the individual pages of a process. So use *pager*, rather than *swapper*, in connection with demand paging.

Basic Concepts

When a process is to be swapped in, the pager guesses which pages will be used before the process is swapped in again. Instead of swapping in a whole process, the pager brings only those necessary pages into memory, decreasing the swap time and the amount of physical memory needed.

With this scheme, we need some form of hardware support to distinguish between the pages that are in memory and the pages that are on the disk.

The *Valid-Invalid bit* scheme can be used for this purpose (Figure)

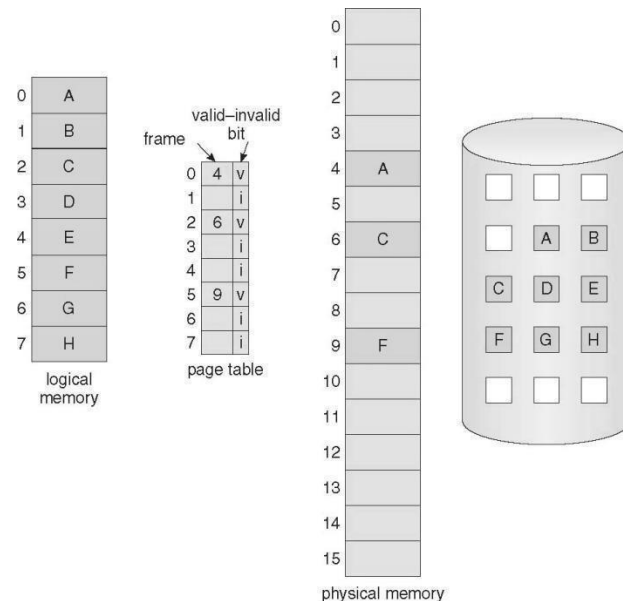


Figure: Page table when some pages are not in Main Memory

When this bit is set to "valid" the associated page is both legal and in memory.

If the bit is set to "invalid," the page either is not valid (that is, not in the logical address space of the process) or is valid but is currently on the disk.

Problem: If the process tries to access a page that was not brought into memory?

Sol: Access to a page marked invalid causes a *Page-Fault Trap*.

The paging hardware, in translating the address through the page table, will notice that the invalid bit is set, causing a trap to the operating system. This trap is the result of the operating system's failure to bring the desired page into memory.

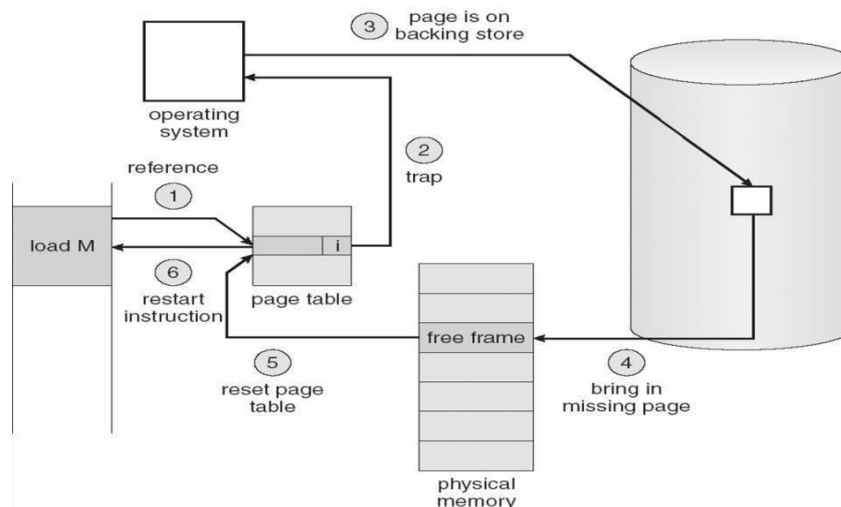


FIG: Steps in Handling a Page Fault

The procedure for handling this page fault is:

1. We check an internal table (usually kept with the process control block) for this process to determine whether the reference was a valid or an invalid memory access.
2. If the reference was invalid, we terminate the process. If it was valid, but we have not yet brought in that page, we now page it in.
3. We find a free frame (by taking one from the free-frame list).

4. We schedule a disk operation to read the desired page into the newly allocated frame.
5. When the disk read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
6. We restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.

Pure Demand Paging: Never bring a page into memory until it is required.

The hardware to support demand paging is the same as the hardware for paging and swapping:

- *Page table:* This table has the ability to mark an entry invalid through a valid-invalid bit or special value of protection bits.

- *Secondary memory:* This memory holds those pages that are not present in main memory.

The secondary memory is usually a high-speed disk. It is known as the swap device, and the section of disk used for this purpose is known as *Swap Space*.

A crucial requirement for demand paging is the need to be able to restart

Advantages:

- Demand paging, as opposed to loading all pages immediately:
- Only loads pages that are demanded by the executing process.
- As there is more space in main memory, more processes can be loaded reducing context switching time which utilizes large amounts of resources.
- Less loading latency occurs at program startup, as less information is accessed from secondary storage and less information is brought into main memory.
- As main memory is expensive compared to secondary memory, this technique helps significantly reduce the bill of material (BOM) cost in smart phones for example. Symbian OS had this feature.

Disadvantages:

- Individual programs face extra latency when they access a page for the first time.
- Low-cost, low-power embedded systems may not have a memory management unit that supports page replacement.
- Memory management with page replacement algorithms becomes slightly more complex.
- Possible security risks, including vulnerability to timing attacks
- Thrashing which may occur due to repeated page faults.

Performance of Demand Paging

Demand paging can significantly affect the performance of a computer system.

Let's compute the Effective Access Time for a demand-paged memory.

For most computer systems, the memory-access time, denoted ma , ranges from 10 to 200 nanoseconds.

As long as we have no page faults, the effective access time is equal to the memory access time. If, however, a page fault occurs, we must first read the relevant page from disk and then access the desired word.

Let p be the probability of a page fault ($0 \leq p \leq 1$). We would expect p to be close to zero—that is, we would expect to have only a few page faults.

The Effective Access Time is then

$$\text{Effective Access Time} = (1 - p) \times ma + p \times \text{page fault time.}$$

To compute the effective access time, we must know how much time is needed to service a page fault. A page fault causes the following sequence to occur:

1. Trap to the operating system.
2. Save the user registers and process state.
3. Determine that the interrupt was a page fault. '
4. Check that the page reference was legal and determine the location of the page on the disk.
5. Issue a read from the disk to a free frame:
 - a. Wait in a queue for this device until the read request is serviced.
 - b. Wait for the device seek and /or latency time.
 - c. Begin the transfer of the page to a free frame.
6. While waiting, allocate the CPU to some other user (CPU scheduling, optional).
7. Receive an interrupt from the disk I/O subsystem (I/O completed).
8. Save the registers and process state for the other user (if step 6 is executed).
9. Determine that the interrupt was from the disk.
10. Correct the page table and other tables to show that the desired page is now in memory.
11. Wait for the CPU to be allocated to this process again.
12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction.

In any case, we are faced with three major components of the Page-Fault Service Time:

1. Service the page-fault interrupt.
2. Read in the page.
3. Restart the process.

The first and third tasks can be reduced, with careful coding, to several hundred instructions.

These tasks may take from 1 to 100 microseconds each.

The page-switch time will probably be close to 8 milliseconds.

Since

A typical hard disk has an average latency of 3 milliseconds,
a seek of 5 milliseconds, and
a transfer time of 0.05 milliseconds.

Thus, the total paging time is about 8 milliseconds, including hardware and software time.

With an average page-fault service time of 8 milliseconds and a memory access time of 200 nanoseconds, the effective access time in nanoseconds is

$$\begin{aligned} \text{Effective Access Time} &= (1 - p) \times (200) + p (8 \text{ milliseconds}) \\ &= (1 - p) \times 200 + p \times 8,000,000 \\ &= 200 + 7,999,800 \times p. \end{aligned}$$

Effective access time is directly proportional to the page-fault rate.

If one access out of 1,000 causes a page fault, the effective access time is 8.2 microseconds. The computer will be slowed down by a factor of 40 because of demand paging. If we want performance degradation to be less than 10 percent, we need to keep the probability of page faults at the following level:

$$\begin{aligned} 220 &> 200 + 7,999,800 \times p, \\ 20 &> 7,999,800 \times p, \\ p &< 0.0000025. \end{aligned}$$

That is, to keep the slowdown due to paging at a reasonable level, we can allow fewer than one memory access out of 399,990 to page-fault. In sum, it is important to keep the page-fault rate low in a demand-paging system. Otherwise, the effective access time increases, slowing process execution dramatically.

An additional aspect of demand paging is the handling and overall use of swap space. Disk I/O to swap space is generally faster than that to the file system. It is a faster file system because swap space is allocated in much larger blocks, and file lookups and indirect allocation methods are not used.

The system can therefore gain better paging throughput by copying an entire file image into the swap space at process startup and then performing demand paging from the swap space.

Mobile operating systems typically do not support swapping.

Page Replacement

If we increase our degree of multiprogramming, we are over-allocating memory.

Ex:

If we run six processes,

each of which is ten pages in size

but actually uses only five pages,

we have higher CPU utilization and throughput, with ten frames to spare.

Over-allocation of memory manifests itself as follows. While a user process is executing, a page fault occurs.

Operating System determines where the desired page is residing on the disk but then finds that there are *no* free frames on the free-frame list; all memory is in use (Figure).

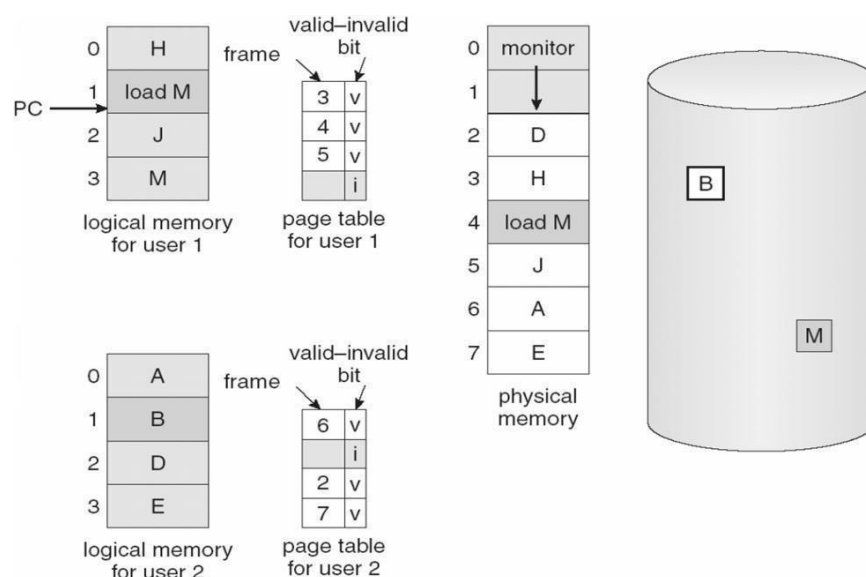


Figure: Need for page replacement.

Operating System has several options at this point. It could terminate the user process. However, demand paging is the operating system's attempt to improve the computer system's utilization and throughput.

Basic Page Replacement

Page replacement takes the following approach.

If no frame is free, we find one that is not currently being used and free it.

We can free a frame by writing its contents to swap space and changing the page table (and all other tables) to indicate that the page is no longer in memory (Figure).

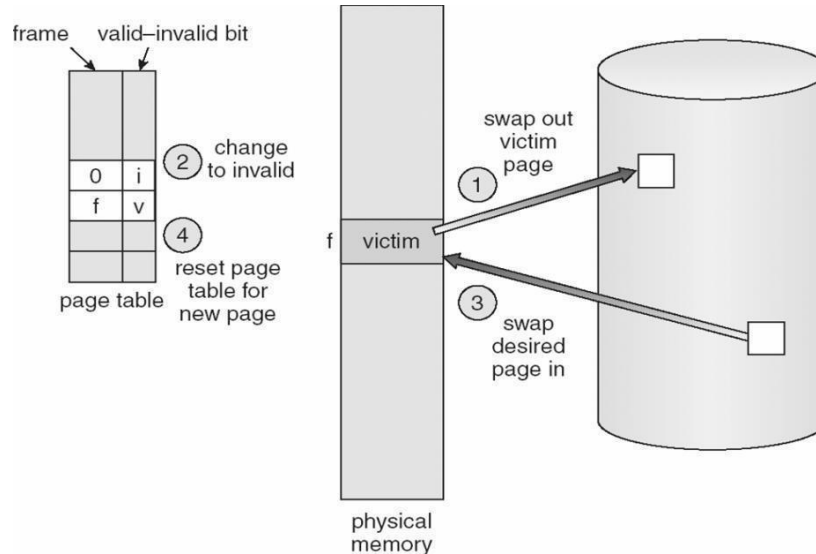


Figure: Page Replacement.

We can now use the freed frame to hold the page for which the process faulted. We modify the page-fault service routine to include page replacement:

1. Find the location of the desired page on the disk.
2. Find a free frame:
 - a. If there is a free frame, use it.
 - b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
 - c. Write the victim frame to the disk; change the page and frame tables accordingly.
3. Read the desired page into the newly freed frame; change the page and frame tables.
4. Restart the user process.

Problem: If no frames are free, *two* page transfers (one out and one in) are required. This situation effectively doubles the page-fault service time and increases the effective access time

Solution: We can reduce this overhead by using a *Modify Bit (or Dirty Bit)*.

When this scheme is used, each page or frame has a modify bit associated with it in the hardware. The modify bit for a page is set by the hardware whenever any word or byte in the page is written into, indicating that the page has been modified.

When we select a page for replacement, we examine its modify bit.

- If the bit is set, we know that the page has been modified since it was read in from the disk. In this case, we must write that page to the disk.
- If the modify bit is not set, however, the page has *not* been modified since it was read into memory.

Page replacement is basic to demand paging. It completes the separation between logical memory and physical memory

We must solve two major problems to implement demand paging:

We must develop:

- Frame-Allocation Algorithm and
- Page-Replacement Algorithm.

If we have multiple processes in memory, we must decide how many frames to allocate to each process. Further, when page replacement is required, we must select the frames that are to be replaced.

We evaluate an algorithm by running it on a particular string of memory references and computing the number of page faults. The string of memory references is called a *Reference String*.

Ex: If we trace a particular process, we might record the following address sequence:

0100, 0432, 0101, 0612, 0102, 0103, 0104, 0101, 0611, 0102, 0103,
0104, 0101, 0610, 0102, 0103, 0104, 0101, 0609, 0102, 0105

At 100 bytes per page, this sequence is reduced to the following reference string:

1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1

To determine the number of page faults for a particular reference string and page-replacement algorithm, we also need to know the number of page frames available.

Obviously, as the number of frames available increases, the number of page faults decreases.

For the reference string considered previously, for example, if we had three or more frames, we would have only three faults—one fault for the first reference to each page.

Next, with only one frame available, we would have a replacement with every reference, resulting in 11 faults. In general, we expect a curve such as that in Figure

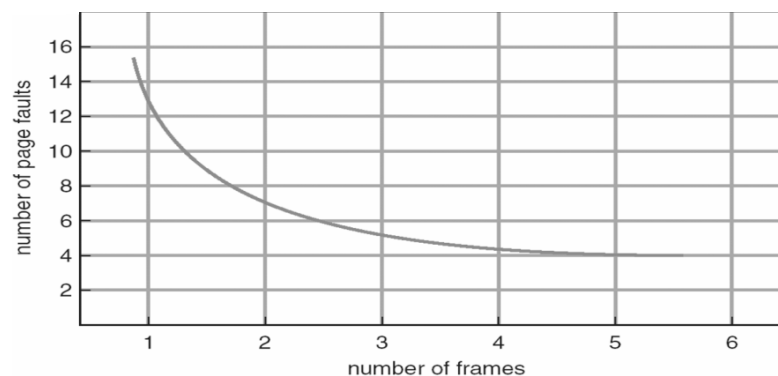


Figure: Graph of page faults versus number of frames.

FIFO Page Replacement

The simplest page-replacement algorithm is a First-In, First-Out (FIFO) algorithm.

A FIFO replacement algorithm associates with each page the time when that page was brought into memory. When a page must be replaced, the oldest page is chosen.

Note: It is not strictly necessary to record the time when a page is brought in.

We can create a FIFO queue to hold all pages in memory. We replace the page at the *Head Of The Queue*. When a page is brought into memory, we insert it at the *Tail Of The Queue*.

For our example reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Our three frames are initially empty.

The first three references (7,0,1) cause page faults and are brought into these empty frames.

The next reference (2) replaces page 7, because page 7 was brought in first.

Since 0 is the next reference and 0 is already in memory, we have no fault for this reference.

The first reference to 3 results in replacement of page 0, since it is now first in line. Because of this replacement, the next reference, to 0, will fault. Page 1 is then replaced by page 0. This process continues as shown in Figure. Every time a fault occurs, we show which pages are in our three frames.

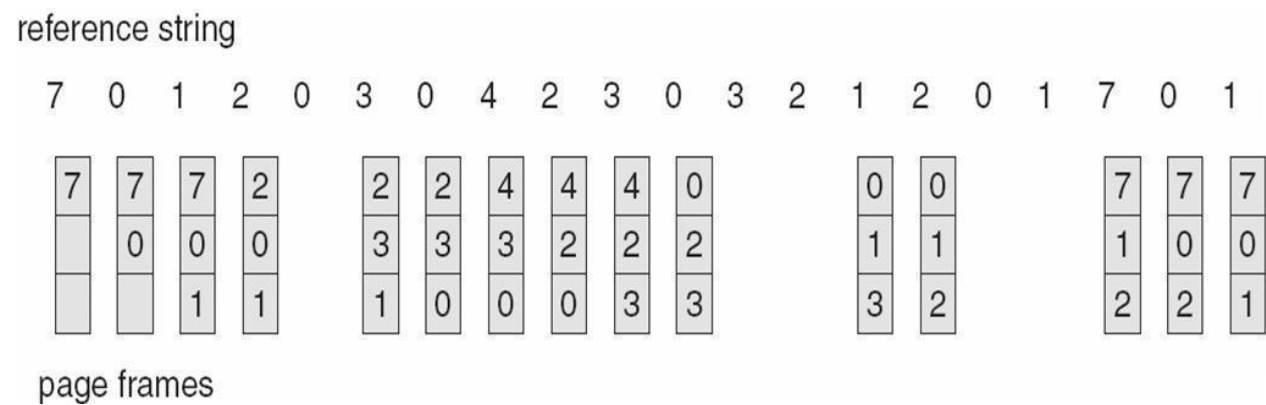


Figure: FIFO Page-Replacement Algorithm.

Number of Page Faults = 15.
Number of Page Replacements = 12

FIFO

	0	1	2	3	0	1	4	0	1	2	3	4
A	△ 0	0	0	△ 3	3	3	△ 4	4	4	4	4	4
B		△ 1	1	1	△ 0	0	0	0	0	△ 2	2	2
C			△ 2	2	2	△ 1	1	1	1	1	△ 3	3

△ = page fault

Advantage:

- Easy to understand and program.
- Correct Execution

Disadvantage:

- Performance is not always good.
- Slows process execution.
- After we replace an active page with a new one, a fault occurs almost immediately to retrieve the active page. Some other page will need to be replaced to bring the active page back into memory. Thus, a bad replacement choice increases the page-fault rate and

To illustrate the problems that are possible with a FIFO page-replacement algorithm., we consider the following reference string: 1,2,3,4,1,2,5,1,2,3,4,5

Figure shows the curve of page faults for this reference string versus the number of available frames.

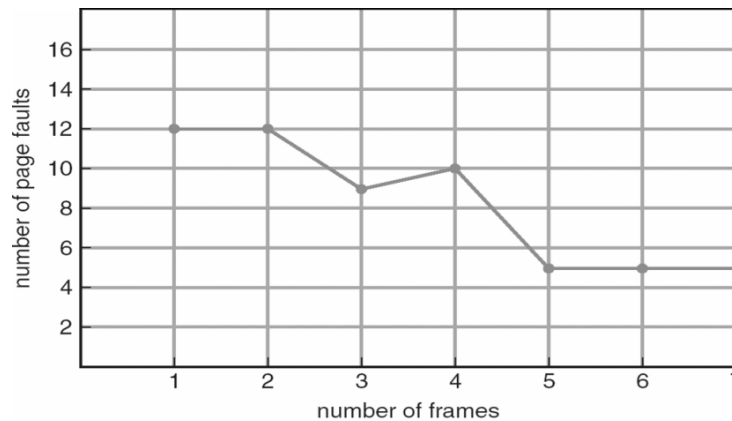


Figure: Page-fault curve for FIFO replacement on a reference string.

Number of faults for four frames (ten) is *greater* than the number of faults for three frames (nine)

This most unexpected result is known as *Belady's Anomaly*

For some page-replacement algorithms, the page-fault rate may *increase* as the number of allocated frames increases.

Optimal Page Replacement

One result of the discovery of Belady's anomaly was the search for an *Optimal Page-Replacement Algorithm*. An optimal page-replacement algorithm has the lowest page-fault rate of all algorithms and will never suffer from Belady's anomaly. Such an algorithm does exist and has been called OPT or MIN.

It is simply this:

“Replace the page that will not be used for the longest period of time.”

Use of this Page-Replacement Algorithm guarantees the lowest possible page fault rate for a fixed number of frames.

Ex: Our sample reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

The first three references cause faults that fill the three empty frames.

The reference to page 2 replaces page 7, because 7 will not be used until reference 18

Page 0 will be used at 5, and page 1 at 14.

The reference to page 3 replaces page 1, as page 1 will be the last of the three pages in memory to be referenced again.

Optimal page-replacement algorithm would yield nine page faults, as shown in Figure

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2	2	2	2	2	7
	0	0	0	0	4	0	0	0
		1	1	3	3	3	1	1

page frames

With only nine page faults, optimal replacement is much better than a FIFO algorithm, which resulted in fifteen faults.

Number of Page Faults = 9

Number of Page Replacements = 6

Optimal Page Replacement Policy

	0	1	2	3	0	1	4	0	1	2	3	4
A	<u>0</u>	0	0	0	0	0	0	0	0	<u>2</u>	2	2
B		<u>1</u>	1	1	1	1	1	1	1	1	<u>3</u>	3
C			<u>2</u>	<u>3</u>	3	3	<u>4</u>	4	4	4	4	4

⬅ = page fault

Advantage: If we ignore the first three, which all algorithms must suffer, then optimal replacement is twice as good as FIFO replacement.

Disadvantage: This algorithm is difficult to implement, because it requires future knowledge of the reference string.

LRU Page Replacement

The key distinction between the FIFO and OPT algorithms (other than looking backward versus forward in time) is that the FIFO algorithm uses the time when a page was brought into memory, whereas the OPT algorithm uses the time when a page is to be used.

If we use the recent past as an approximation of the near future, then we can replace the page that *has not been used* for the longest period of time. This approach is the *Least-Recently-Used (LRU) Algorithm*.

LRU replacement associates with each page the time of that page's last use. When a page must be replaced, LRU chooses the page that has not been used for the longest period of time.

Ex: Our sample reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1
	0	0	0		0		0	0	3	3		3		0		0
		1	1		3		3	2	2	2		2		2		7

page frames

Figure : LRU Page-Replacement Algorithm.

The LRU algorithm produces 12 faults. Notice that the first 5 faults are the same as those for optimal replacement.

When the reference to page 4 occurs, however, LRU replacement sees that, of the three frames in memory, page 2 was used least recently.

Thus, the LRU algorithm replaces page 2, not knowing that page 2 is about to be used. When it then faults for page 2, the LRU algorithm replaces page 3, since it is now the least recently used of the three pages in memory.

Number of Page Faults = 12

Number of Page Replacements = 9

Least Recently Used

	0	1	2	3	0	1	4	0	1	2	3	4
A	0	0	0	3	3	3	4	4	4	2	2	2
B		1	1	1	0	0	0	0	0	0	3	3
C			2	2	2	1	1	1	1	1	1	4

▲ = page fault

The problem is to determine an order for the frames defined by the time of last use.

Two implementations are feasible:

- *Counters.* In the simplest case, we associate with each page-table entry a time-of-use field and add to the CPU a logical clock or counter. The clock is incremented for every memory reference.

Whenever a reference to a page is made, the contents of the clock register are copied to the time-of-use field in the page-table entry for that page. In this way, we always have the "time" of the last reference to each page. We replace the page with the smallest time value.

- *Stack.* Another approach to implementing LRU replacement is to keep a stack of page numbers. Whenever a page is referenced, it is removed from the stack and put on the top. In this way, the most recently used page is always at the top of the stack and the least recently used page is always at the bottom

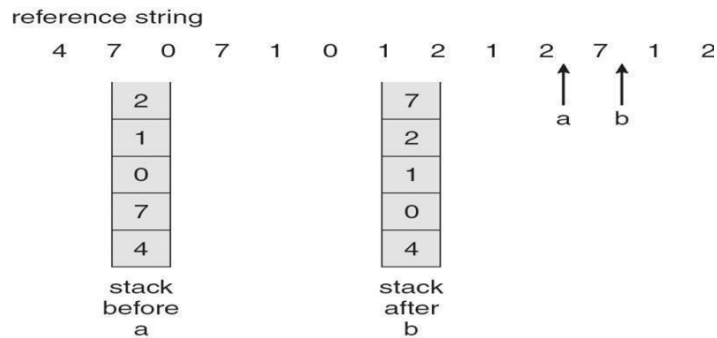


FIG: Use Of A Stack to Record The Most Recent Page References

This approach is particularly appropriate for software or microcode implementations of LRU replacement.

Like optimal replacement, LRU replacement does not suffer from Belady's anomaly.

Both belong to a class of page-replacement algorithms, called Stack algorithms, that can never exhibit Belady's anomaly.

LRU-Approximation Page Replacement

Few computer systems provide sufficient hardware support for true LRU page replacement. Some systems provide no hardware support, and other page replacement algorithms (such as a FIFO algorithm) must be used.

Many systems provide some help, however, in the form of a *Reference Bit*.

The reference bit for a page is set by the hardware whenever that page is referenced (either a read or a write to any byte in the page).

Reference bits are associated with each entry in the page table.

Initially, all bits are cleared (to 0) by the operating system. As a user process executes, the bit associated with each page referenced is set (to 1) by the hardware.

After some time, we can determine which pages have been used and which have not been used by examining the reference bits, although we do not know the *order* of use. This information is the basis for many page-replacement algorithms that approximate LRU replacement.

Additional-Reference-Bits Algorithm

Above LRU Approximation is giving only partial information, which pages are used; but it doesn't know about any order of use. That's why we lead to:

- Additional-Reference-Bits Algorithm
- Second-Chance Algorithm
- Enhanced Second-Chance Algorithm

We can gain additional ordering information by recording the reference bits at regular intervals.

We can keep an 8-bit byte for each page in a table in memory.

At regular intervals (say, every 100 milliseconds), a timer interrupt transfers control to the operating system.

The operating system shifts the reference bit for each page into the high-order bit of its 8-bit byte, shifting the other bits right by 1 bit and discarding the low-order bit. These 8-bit shift registers contain the history of page use for the last eight time periods.

8-bit Reference

→	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	Page 1 No Execution i.e., Complete
	0	1	0	0	0	0	0	Page 2 Execution
	0	0	0	0	0	0	0	Page 3 No Execution
	0	0	0	0	1	0	0	Page 4 Execution

Before starting execution it is zero. After one clock cycle there is execution

Second-Chance Algorithm

The basic algorithm of second-chance replacement is a FIFO replacement algorithm.

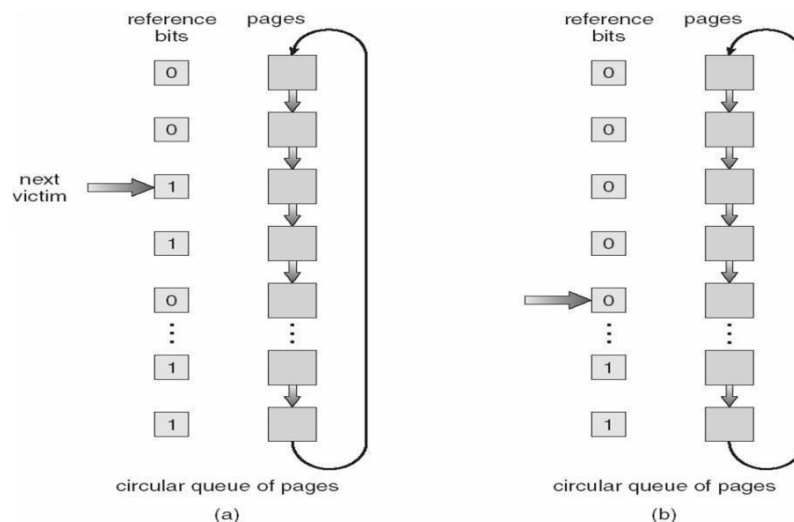


Figure: Second-chance (clock) page-replacement algorithm.

When a page has been selected, however, we inspect its reference bits bit.

- If the Reference bit value is 0, we proceed to replace this page;
- If the Reference bit is set to 1, we give the page a second chance and move on to select the next FIFO page.

When a page gets a second chance, its reference bit is cleared, and its arrival time is reset to the current time. Thus, a page that is given a second chance will not be replaced until all other pages have been replaced (or given second chances).

In addition, if a page is used often enough to keep its reference bit set, it will never be replaced. One way to implement the second-chance algorithm (sometimes referred to as the *dock* algorithm) is as a circular queue.

A pointer (that is, a hand on the clock) indicates which page is to be replaced next. When a frame is needed, the pointer advances until it finds a page with a 0 reference bit. As it advances, it clears the reference bits (Figure).

Once a victim page is found, the page is replaced, and the new page is inserted in the circular queue in that position.

In the worst case, when all bits are set, the pointer cycles through the whole queue, giving each page a second chance. It clears all the reference bits before selecting the next page for replacement. Second-chance replacement degenerates to FIFO replacement if all bits are set.

Enhanced Second-Chance Algorithm

We can enhance the second-chance algorithm by considering the reference bit and the modify bit as an ordered pair.

With these two bits, we have the following four possible classes:

- (0, 0) neither recently used nor modified—best page to replace
- (0, 1) not recently used but modified—not quite as good, because the page will need to be written out before replacement
- (1, 0) recently used but clean—probably will be used again soon
- (1, 1) recently used and modified—probably will be used again soon, and the page will be need to be written out to disk before it can be replaced

Each page is in one of these four classes.

When page replacement is called for, we use the same scheme as in the clock algorithm; but instead of examining whether the page to which we are pointing has the reference bit set to 1, we examine the class to which that page belongs.

We replace the first page encountered in the lowest nonempty class. Notice that we may have to scan the circular queue several times before we find a page to be replaced.

Major difference between Enhanced Second-Chance Algorithm algorithm and the simpler clock algorithm is that here we give preference to those pages that have been modified to reduce the number of I/Os required.

Counting-Based Page Replacement

There are many other algorithms that can be used for page replacement.

Ex: We can keep a counter of the number of references that have been made to each page and develop the following two schemes.

- The *Least Frequently Used (LFU) page-replacement algorithm* requires that the page with the smallest count be replaced.

The reason for this selection is that an actively used page should have a large reference count.

A problem arises, however, when a page is used heavily during the initial phase of a process but then is never used again. Since it was used heavily, it has a large count and remains in memory even though it is no longer needed.

One solution is to shift the counts right by 1 bit at regular intervals, forming an exponentially decaying average usage count.

- The *Most Frequently Used (MFU) page-replacement algorithm* is based on the argument that the page with the smallest count was probably just brought in and has yet to be used.

The implementation of these algorithms is expensive, and they do not approximate OPT replacement well.

Page-Buffering Algorithms

Other procedures are often used in addition to a specific page-replacement algorithm.

Ex: Systems commonly keep a pool of free frames.

When a page fault occurs, a victim frame is chosen as before. However, the desired page is read into a free frame from the pool before the victim is written out. This procedure allows the process to restart as soon as possible, without waiting for the victim page to be written out. When the victim is later written put, its frame is added to the free-frame pool.

An expansion of this idea is to maintain a list of modified pages. Whenever the paging device is idle, a modified page is selected and is written to the disk. Its modify bit is then reset. This

scheme increases the probability that a page will be clean when it is selected for replacement and will not need to be written out.

Applications and Page Replacement

In certain cases, applications accessing data through the operating system's virtual memory perform, worse than if the operating system provided no buffering at all.

Ex: Database, which provides its own memory management and I/O buffering.

If the operating system is buffering I/O, and the application is doing so as well, then twice the memory is being used for a set of I/O.

Ex: Data warehouses frequently perform massive sequential disk reads, followed by computations and writes. The LRU algorithm would be removing old pages and preserving new ones, while the application would more likely be reading older pages than newer ones (as it starts its sequential reads again). Here, MFU would actually be more efficient than LRU.

Frame Allocation

How do we allocate the fixed amount of free memory among the various processes?

If we have 93 free frames and two processes, how many frames does each process get?

The simplest case is the single-user system.

Consider a single-user system with 128 KB of memory composed of pages 1 KB in size. This system has 128 frames. The operating system may take 35 KB, leaving 93 frames for the user process.

Under pure demand paging, all 93 frames would initially be put on the free-frame list. When a user process started execution, it would generate a sequence of page faults. The first 93 page faults would all get free frames from the free-frame list. When the free-frame list was exhausted, a page-replacement

algorithm would be used to select one of the 93 in-memory pages to be replaced with the 94th, and so on. When the process terminated, the 93 frames would once again be placed on the free-frame list.

Minimum Number of Frames

Strategies for the allocation of frames are constrained in various ways.

We cannot, for example, allocate more than the total number of available frames (unless there is page sharing). We must also allocate at least a minimum number of frames.

One reason for allocating at least a minimum number of frames involves performance. Obviously, as the number of frames allocated to each process decreases, the page-fault rate increases, slowing process execution.

When a page fault occurs before an executing instruction is complete, the instruction must be restarted. Consequently, we must have enough frames to hold all the different pages that any single instruction can reference.

Allocation Algorithms

The easiest way to split m frames among n processes is to give everyone an equal share, m/n frames.

Ex: If there are 93 frames and five processes, each process will get 18 frames. The leftover three frames can be used as a free-frame buffer pool. This scheme is called *Equal Allocation*.

An alternative is to recognize that various processes will need differing amounts of memory.

Ex: Consider a system with a 1-KB frame size.

If a small student process of 10 KB and

an interactive database of 127 KB are the only two processes running in a system with 62 free frames, it does not make much sense to give each process 31 frames.

The student process does not need more than 10 frames, so the other 21 are, strictly speaking, wasted.

To solve this problem, we can use proportional allocation, in which we allocate available memory to each process according to its size.

Let the size of the virtual memory for process p_i be s_i and define

Then, if the total number of available frames is m , we allocate a_i frames to process p_i ,

where a_i is approximately $a_i = \frac{s_i}{s} \times m$.

Of course, we must adjust each a_i to be an integer that is greater than the minimum number of frames required by the instruction set, with a sum not exceeding m .

For proportional allocation, we would split 62 frames between two processes, one of 10 pages and one of 127 pages, by allocating 4 frames and 57 frames, respectively, since

$$10/137 \times 62 \approx 4, \text{ and}$$

$$127/137 \times 62 \approx 57.$$

In this way, both processes share the available frames according to their "needs," rather than equally.

In both equal and proportional allocation, the allocation may vary according to the multiprogramming level. If the multiprogramming level is increased, each process will lose some frames to provide the memory needed for the new process. Conversely, if the multiprogramming level decreases, the frames that were allocated to the departed process can be spread over the remaining processes.

Notice that, with either equal or proportional allocation, a high-priority process is treated the same as a low-priority process. By its definition, however, we may want to give the high-priority process more memory to speed its execution, to the detriment of low-priority processes.

Global versus Local Allocation

Another important factor in the way frames are allocated to the various processes is page replacement. With multiple processes competing for frames, we can classify page-replacement algorithms into two broad categories:

- Global replacement and
- Local replacement.

Local replacement: It requires that each process select from only its own set of allocated frames. Consider an allocation scheme where we allow high-priority processes to select frames from low-priority processes for replacement.

A process can select a replacement from among its own frames or the frames of any lower-priority process. This approach allows a high-priority process to increase its frame allocation at the expense of a low-priority process.

With a Local Replacement strategy, the number of frames allocated to a process does not change.

Under local replacement, the set of pages in memory for a process is affected by the paging behavior of only that process.

Local replacement might hinder a process, however, by not making available to it other, less used pages of memory.

Global replacement: It allows a process to select a replacement frame from the set of all frames, even if that frame is currently allocated to some other process; that is, one process can take a frame from another.

Ex: Consider an allocation scheme where we allow high-priority processes to select frames from low-priority processes for replacement.

With global replacement, a process may happen to select only frames allocated to other processes, thus increasing the number of frames allocated to it (assuming that other processes do not choose its frames for replacement).

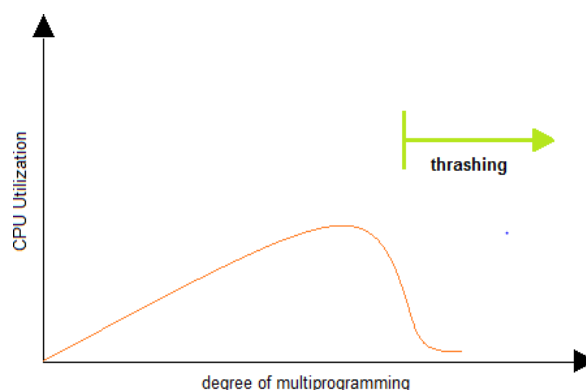
Thus, global replacement is a more common method that generally results in greater system throughput

THRASHING

- A process that is spending more time paging than executing is said to be thrashing.
- In other words it means, that the process doesn't have enough frames to hold all the pages for its execution, so it is swapping pages in and out very frequently to keep executing. Sometimes, the pages which will be required in the near future have to be swapped out.

Initially when the CPU utilization is low, the process scheduling mechanism, to increase the level of multiprogramming loads multiple processes into the memory at the same time, allocating a limited amount of frames to each process.

As the memory fills up, process starts to spend a lot of time for the required pages to be swapped in, again leading to low CPU utilization because most of the processes are waiting for pages. Hence the scheduler loads more processes to increase CPU utilization, as this continues at a point of time the complete system comes to a stop.



Causes of Thrashing

It results in severe performance problems.

- 1) If CPU utilization is too low then we increase the degree of multiprogramming by introducing a new process to the system. A global page replacement algorithm is used. The CPU scheduler sees the decreasing CPU utilization and increases the degree of multiprogramming.
- 2) CPU utilization is plotted against the degree of multiprogramming.
- 3) As the degree of multiprogramming increases, CPU utilization also increases.
- 4) If the degree of multiprogramming is increased further, thrashing sets in and CPU utilization drops sharply.
- 5) So, at this point, to increase CPU utilization and to stop thrashing, we must decrease the degree of multiprogramming.

How to prevent Thrashing

- We must provide a process with as many frames as it needs. Several techniques are used.
- The Working of Set Model (Strategy) It starts by looking at how many frames a process is actually using. This defines the locality model.
- Locality Model It states that as a process executes, it moves from locality to locality.
- A locality is a set of pages that are actively used together.
A program is generally composed of several different localities which overlap.

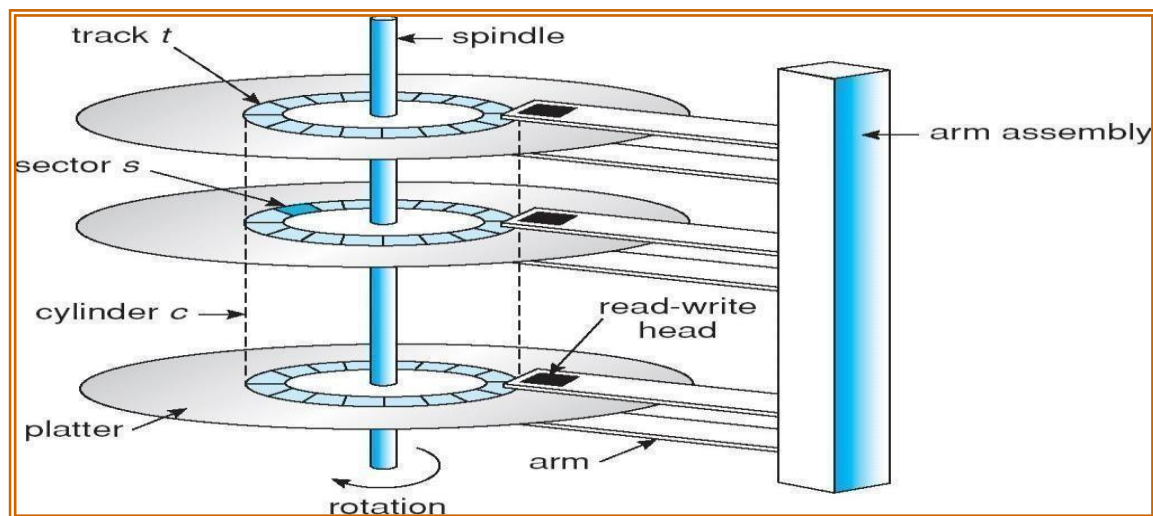
Overview of Mass Storage Structure

Magnetic disks provide bulk of secondary storage of modern computers

- Drives rotate at 60 to 200 times per second
- Transfer rate is rate at which data flow between drive and computer
- Positioning time (random-access time) is time to move disk arm to desired cylinder (seek time) and time for desired sector to rotate under the disk head (rotational latency)
- Head crash results from disk head making contact with the disksurface Disks can be removable

Drive attached to computer via I/O bus

- Busses vary, including SATA, USB, Fibre Channel, SCSI
- Host controller in computer uses bus to talk to disk controller built into drive or storage array



Moving-head Disk Mechanism

- **Magnetic tape**
 - Was early secondary-storage medium
 - Relatively permanent and holds large quantities of data
 - Access time slow
 - Random access ~1000 times slower than disk
 - Mainly used for backup, storage of infrequently-used data, transfer medium between systems
 - Kept in spool and wound or rewound past read-write head
 - Once data under head, transfer rates comparable to disk
 - 20-200GB typical storage

Disk Structure

- Disk drives are addressed as large 1-dimensional arrays of *logical blocks*, where the logical block is the smallest unit of transfer.
- The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially.
 - Sector 0 is the first sector of the first track on the outermost cylinder.
 - Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.

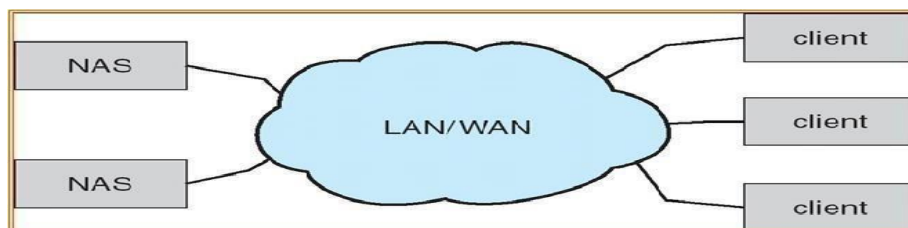
Disk Attachment

- Host-attached storage accessed through I/O ports talking to I/O busses
- SCSI itself is a bus, up to 16 devices on one cable, **SCSI initiator** requests operation and **SCSI targets** perform tasks
 - Each target can have up to 8 **logical units** (disks attached to device controller)
- FC is high-speed serial architecture

- Can be switched fabric with 24-bit address space – the basis of storage area networks (SANs) in which many hosts attach to many storage units
- Can be arbitrated loop (FC-AL) of 126 devices

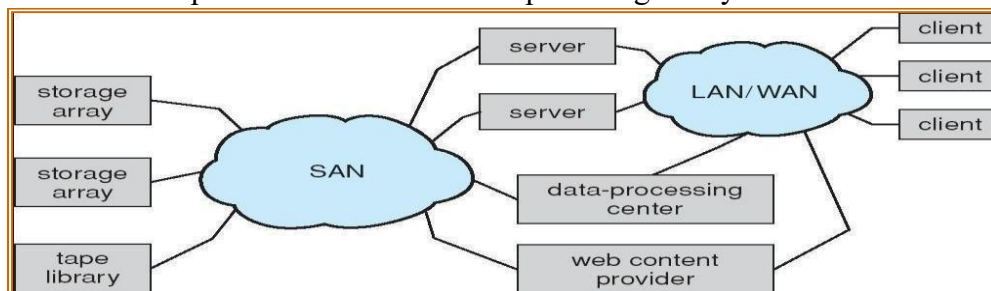
Network-Attached Storage

- Network-attached storage (NAS) is storage made available over a network rather than over a local connection (such as a bus)
- NFS and CIFS are common protocols
- Implemented via remote procedure calls (RPCs) between host and storage. New SCSI protocol uses IP network to carry the SCSI protocol



Storage Area Network

- Common in large storage environments (and becoming more common)
- Multiple hosts attached to multiple storage arrays - flexible



HDD Scheduling

HDD scheduling refers to the methods used by the OS to decide the order in which disk I/O requests are serviced.

Because hard disks have moving read/write heads, the performance depends mainly on:

- Seek time → time to move the head to the correct track
- Rotational latency → time for the sector to rotate under the head
- Transfer time → time to transfer data

To minimize seek time and improve throughput, OS uses disk scheduling algorithms.\

First-Come First-Served (FCFS)

- Requests are processed in the order they arrive.
- Simple and fair, but not optimal.
- Long wait time if far-apart tracks appear consecutively.

Pros: No starvation

Cons: Long response time

Shortest Seek Time First (SSTF)

- Selects the request with the least seek time from the current head position.
- Like SJF in CPU scheduling.

Pros: Better performance than FCFS

Cons: May cause starvation (far requests ignored)

SCAN (Elevator Algorithm)

- Head moves like an elevator: in one direction, services all requests, then reverses.
- Reduces head movement.

Pros: Fair, no starvation

Cons: Returning direction may waste time

C-SCAN (Circular SCAN)

- Head moves in only one direction.
- When it reaches the end, it returns to the beginning without servicing any request.

Pros: More uniform wait time

Cons: Slightly higher overhead

LOOK and C-LOOK

Improved versions of SCAN and C-SCAN

- Head goes only as far as the final request, not to the disk end.

LOOK → Reverses at last request

C-LOOK → One-direction circular movement

Selection Between Algorithms

Algorithm	Performance	Starvation	Use Case
FCFS	Low	No	Simple systems
SSTF	Medium	Yes	Moderate load
SCAN	High	No	Heavy load
C-SCAN	High	No	Frequently used in OS
LOOK / C-LOOK	Best practical	No	Modern OS

Short Answer Questions

1. What is the role of CPU registers in memory access?
2. Why does the CPU stall when accessing main memory?
3. Define cache memory.
4. What is the purpose of base and limit registers?
5. Give an example of how base and limit registers define legal memory access.
6. What is memory protection in operating systems?
7. Define address binding.
8. List the three stages at which address binding can occur.
9. What is compile-time binding?
10. What is load-time binding?
11. What is execution-time binding?
12. Define logical address.
13. Define physical address.
14. What is the function of the Memory Management Unit (MMU)?
15. What is dynamic loading?